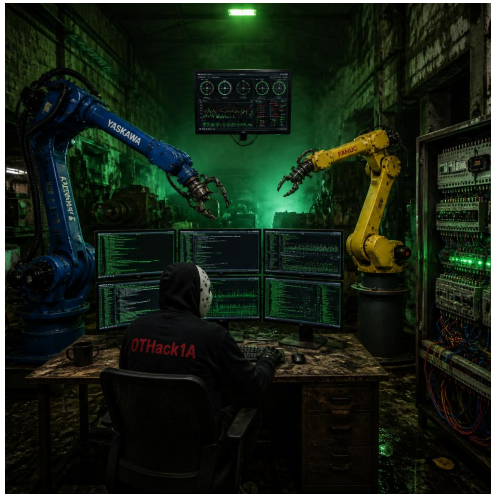




Docker

Dal Container all'Intelligenza Artificiale

Ambiente: Debian 12 • Arch Linux • Docker Engine 26.x

Storia, architettura, best practice, esempi pratici e integrazione con i modelli AI

Creator:

OTHack1A

<https://github.com/OTHack1A>

Documento personale — Solo uso didattico

Docker — Dal Container all'Intelligenza Artificiale

Creator: OTHack1A <https://github.com/OTHack1A>

Documento personale a uso esclusivamente didattico.

Docker® è un marchio registrato di Docker, Inc.

Esempi testati su Debian 12 Bookworm e Arch Linux (Giugno 2026).

Fonti principali:

Docker Docs — <https://docs.docker.com>

Docker Blog — <https://www.docker.com/blog>

Compose Spec — <https://compose-spec.io>

OCI Spec — <https://opencontainers.org>

Indice

Prefazione	5
1 Storia e Origini dei Container	6
1.1 Prima di Docker: le radici della containerizzazione	6
1.1.1 chroot: il capostipite (1979)	6
1.1.2 cgroups e namespace: i mattoni di Docker	7
1.2 La nascita di Docker (2013)	7
1.3 Docker vs Macchine Virtuali	7
1.4 L'ecosistema nel 2026	8
1.5 Docker in cifre: adozione nel mondo reale	9
1.6 Lo standard OCI: perché è importante	9
2 Architettura Interna di Docker	10
2.1 Componenti del sistema Docker	10
2.2 Come funziona <code>docker run</code>	10
2.3 UnionFS e i layer delle immagini	11
2.4 Il registro OCI: Docker Hub e alternative	12
2.5 BuildKit: il motore di build avanzato	12
3 Installazione di Docker	14
3.1 Installazione su Debian 12 (Bookworm) e Kali Linux	14
3.2 Installazione su Arch Linux	15
3.3 Configurazione post-installazione	15
3.4 Docker senza root: modalita' rootless	16
4 Dockerfile: Costruire Immagini	17
4.1 Istruzioni Dockerfile	17
4.2 Dockerfile annotato	17
4.3 Multi-stage build	18
4.4 <code>.dockerignore</code>	19
4.5 Best practice Dockerfile	20
4.6 Ottimizzazione avanzata dei layer	21
4.6.1 L'ordine degli strati conta	21
4.6.2 Analisi visuale dei layer con <code>dive</code>	22
4.7 Dockerfile per linguaggi specifici	22
5 Docker Compose: Stack Multi-Container	24
5.1 Evoluzione di Compose	24
5.2 Struttura di <code>compose.yaml</code>	24
5.3 Comandi Compose essenziali	27
5.4 Watch mode: sviluppo con <code>hot-reload</code>	28
5.5 Profiles: configurazioni condizionali	28

5.6	Variabili d'ambiente e interpolazione	29
6	Networking e Volumi	31
6.1	Reti Docker	31
6.2	Volumi e persistenza dei dati	32
6.3	Docker DNS e service discovery	33
6.4	Port publishing e forwarding	33
6.5	Reverse proxy con Traefik	34
6.6	Volumi avanzati: backup e restore	35
7	Sicurezza in Docker	37
7.1	Il modello di sicurezza di Docker	37
7.2	Hardening dei container	37
7.3	Docker Scout: analisi CVE nelle immagini	38
7.4	Secrets management	38
7.5	Docker Bench for Security	39
8	Docker e Intelligenza Artificiale	40
8.1	Perche' Docker è ideale per l'AI	40
8.2	NVIDIA Container Toolkit e GPU	40
8.3	Eseguire LLM in locale con Docker	41
8.4	Docker GenAI Stack con NVIDIA NIM	42
8.5	Docker AI Catalog	43
8.6	Containerizzare un modello custom	44
9	Sviluppi Futuri ed Ecosistema 2026	46
9.1	Lo stato dell'ecosistema nel 2026	46
9.1.1	containerd 2.0: il nuovo runtime	46
9.1.2	Podman 5.x: alternativa rootless	46
9.2	WebAssembly: la nuova frontiera	47
9.3	Docker e Kubernetes	47
9.4	Tendenze e sviluppi futuri	48
9.5	Sintesi: scegliere il tool giusto nel 2026	49
10	YAML: Il Linguaggio di Docker Compose	50
10.1	Che cos'è YAML	50
10.2	Sintassi YAML: regole fondamentali	50
10.2.1	Indentazione: la regola più importante	50
10.2.2	Tipi di dati scalari	51
10.2.3	Strutture composite: mappe e sequenze	52
10.2.4	Funzionalità avanzate YAML	53
10.3	Perché YAML in Docker Compose	54
10.4	Struttura di compose.yaml spiegata dall'interno	54
10.5	Errori YAML comuni e come evitarli	56
10.6	Validare un file YAML	57
10.7	YAML in altri contesti Docker	58
11	Cybersecurity e Docker	60

11.1 Threat model: superficie di attacco	60
11.2 Container escape: tecniche e prevenzione	60
11.2.1 Il pericolo del socket Docker	61
11.3 Immagini sicure: supply chain security	62
11.3.1 Scegliere immagini base sicure	63
11.4 Runtime security: seccomp, AppArmor, SELinux	63
11.5 Scanning e auditing continuo	65
11.6 Network security: micro-segmentazione	66
11.7 Secrets management avanzato	67
11.8 Incident response: cosa fare dopo una compromissione	68
12 Comandi Avanzati e Opzioni Raramente Usate	70
12.1 docker build: opzioni avanzate	70
12.2 docker run: flag raramente usati	71
12.3 docker inspect: estrazione informazioni avanzata	73
12.4 docker events: monitoraggio in tempo reale	74
12.5 docker cp, diff e commit	74
12.6 Comandi di sistema e manutenzione	76
12.7 BuildKit: funzionalità avanzate poco conosciute	77
13 Diagnostica e Troubleshooting Avanzato	79
13.1 Metodologia di debug	79
13.2 Container che non si avvia	79
13.3 Problemi di rete	80
13.4 Problemi di performance	81
13.5 Debug avanzato con nsenter	83
13.6 Log aggregation e analisi	83
13.7 Fonti per il troubleshooting	85
A Progetto Completo: Server Apache2 da Zero	86
A.1 Struttura del progetto	86
A.2 Passo 1 — La pagina HTML	86
A.3 Passo 2 — Configurazione Apache2	88
A.4 Passo 3 — Il Dockerfile	89
A.5 Passo 4 — .dockerignore	90
A.6 Passo 5 — compose.yaml	91
A.7 Passo 6 — Build, avvio e test	92
A.8 Passo 7 — Variante con HTTPS (TLS)	93
A.9 Riepilogo: cosa abbiamo costruito	96
B Cheatsheet Comandi Docker	97
B.1 Immagini	97
B.2 Container	98
B.3 Volumi e Reti	98
B.4 Docker Compose	99
B.5 Pulizia sistema	99
C Fonti e Riferimenti	101

C.1 Documentazione ufficiale	101
C.2 Articoli e risorse online	101
C.3 Libri consigliati	101
C.4 Fonti per sicurezza e diagnostica	102
Glossario	103

Elenco delle figure

2.1 Architettura a strati di Docker Engine	10
11.1 Threat model Docker: principali vettori di attacco	60

Elenco delle tabelle

1.1	Cronologia della containerizzazione pre-Docker	6
1.2	Milestone storiche di Docker	7
1.3	Container Docker vs Macchine Virtuali	8
1.4	Ecosistema container nel 2026	8
1.5	Dati di adozione Docker 2025–2026	9
1.6	Standard OCI e loro impatto	9
2.1	Componenti Docker e loro responsabilità	10
4.1	Istruzioni Dockerfile complete	17
5.1	Evoluzione di Docker Compose	24
6.1	Driver di rete Docker	31
6.2	Tipi di mount in Docker	32
8.1	Vantaggi di Docker per workload AI/ML	40
9.1	Tendenze nell’ecosistema container 2026–2028	48
9.2	Guida alla scelta del tool container	49
10.1	YAML vs JSON vs TOML vs INI a confronto	50
10.2	Motivazioni della scelta YAML per Docker Compose	54
11.1	Immagini base e loro profilo di sicurezza	63
A.1	Riepilogo del progetto Apache2 Docker	96
B.1	Comandi per la gestione delle immagini	97
B.2	Comandi per la gestione dei container	98
B.3	Comandi per volumi e reti	98
B.4	Comandi Docker Compose v2/v5 essenziali	99

Prefazione

Questo documento è una guida completa a Docker: dalla sua storia e architettura interna, fino agli scenari avanzati di integrazione con l'intelligenza artificiale generativa e i modelli linguistici di grandi dimensioni.

A chi è rivolto: sviluppatori, amministratori di sistema, ingegneri DevOps e chiunque voglia padroneggiare Docker andando oltre i comandi di base.

Struttura:

1. Storia e origini dei container
2. Architettura interna di Docker
3. Installazione su Debian e Arch Linux
4. Dockerfile: sintassi e best practice
5. Docker Compose: stack multi-container
6. Networking e volumi
7. Sicurezza e hardening
8. Docker e Intelligenza Artificiale
9. Sviluppi futuri e ecosistema 2026

OTHack1A

Capitolo 1

Storia e Origini dei Container

1.1 Prima di Docker: le radici della containerizzazione

L'idea di isolare processi non è nata con Docker. Le sue radici affondano in decenni di evoluzione dei sistemi operativi Unix e Linux.

Tabella 1.1: Cronologia della containerizzazione pre-Docker

Anno	Tecnologia	Contributo
1979	chroot (Unix V7)	Prima forma di isolamento filesystem: cambia la root directory per un processo
1999	FreeBSD Jails	Isolamento completo di processi, rete e filesystem su FreeBSD
2001	Linux VServer	Partizioni virtuali del kernel Linux (patch)
2004	Solaris Zones	Container OS-level maturi su Solaris 10
2005	OpenVZ	Virtualizzazione a livello OS per Linux
2006	Google cgroups	Control groups per limitare risorse CPU/RAM (poi integrati nel kernel 2.6.24, 2008)
2008	LXC 0.1	Linux Containers: primo approccio container moderno usando cgroups + namespace
2011	CloudFoundry Warden	Container per PaaS su LXC
2013	Docker 0.1	La rivoluzione: container semplici, portabili e con un ecosistema completo

1.1.1 chroot: il capostipite (1979)

chroot (change root) è il meccanismo più antico di isolamento su Unix. Cambia la directory root percepita da un processo, impedendogli di accedere a file al di fuori di quella gerarchia.

Listing 1.1: chroot: come funziona ancora oggi

```
1 # Crea un ambiente chroot minimale
2 mkdir -p /tmp/mychroot/{bin,lib,lib64}
3 cp /bin/bash /tmp/mychroot/bin/
4 # Copia le librerie necessarie
5 ldd /bin/bash | grep -o '^[^ ]*' | xargs -I{} cp {} /tmp/mychroot/lib/
6
7 # Entra nell'ambiente chroot
8 sudo chroot /tmp/mychroot /bin/bash
9 # Ora non puoi uscire dal /tmp/mychroot (a meno di exploit)
10 # ls / mostra solo: bin lib lib64
11
12 # Limitazioni di chroot:
13 # - Solo isolamento filesystem, NON rete, processi, IPC
14 # - Root può sempre uscire
```

```
15 # - Non limita le risorse (CPU, RAM)
```

1.1.2 cgroups e namespace: i mattoni di Docker

Docker non ha inventato nulla di nuovo a livello kernel: ha semplicemente reso **facili e accessibili** due tecnologie kernel già esistenti.

Nota

Docker = cgroups (limitazione risorse) + namespace (isolamento) + UnionFS (layer filesystem) + API semplice + ecosistema

1.2 La nascita di Docker (2013)

Docker nasce il **13 marzo 2013** quando Solomon Hykes presenta “*The future of Linux Containers*” alla PyCon US. Il video originale dura appena 5 minuti e mostra la creazione di un container in pochi secondi.

Tabella 1.2: Milestone storiche di Docker

Anno	Versione/Evento	Novità chiave
Mar 2013	Docker 0.1	Primo rilascio pubblico (su LXC)
Nov 2013	Docker 0.6	Docker Hub (registry pubblico)
2014	Docker 1.0	Prima versione production-ready
2014	libcontainer	Docker abbandona LXC per il suo runtime nativo
2015	Docker Compose	Gestione multi-container (da fig)
2015	Docker Swarm	Orchestrazione nativa
2015	OCI fondato	Open Container Initiative: standard aperti
2016	Docker 1.12	Swarm mode integrato nel daemon
2017	Moby Project	Docker diventa open source modulare
2017	containerd 1.0	Runtime separato, donato alla CNCF
2020	Compose v2	Riscrittura in Go, plugin CLI nativo
2022	Docker 20.10 EOL	BuildKit default su Linux
2023	Docker 24.0	SemVer, BuildKit maturo
2024	containerd 2.0	Runtime riscritto, hardening sicurezza
Apr 2025	Compose v1 EOL	docker-compose (Python) rimosso
2025	Compose v5	Go SDK ufficiale, Docker Bake integrato
2026	Docker 26.x	GPU enhanced support, AI Catalog integrato

1.3 Docker vs Macchine Virtuali

Una confusione comune: Docker non è una VM. La differenza è fondamentale a livello architetturale.

Tabella 1.3: Container Docker vs Macchine Virtuali

Aspetto	Container Docker	VM tradizionale
Kernel	Condivide il kernel host	Kernel proprio (Guest OS)
Avvio	Millisecondi	Decine di secondi
Overhead	Minimo (quasi bare metal)	Significativo (hypervisor)
Isolamento	Namespace + cgroups	Hardware virtualization
Dimensione	MB (immagine)	GB (disco virtuale)
Portabilità	Immagine OCI universale	Dipende dall'hypervisor
Sicurezza	Kernel condiviso (rischio)	Isolamento hardware completo
Densità	Centinaia per host	Decine per host

Suggerimento

Quando usare cosa? Container Docker: microservizi, CI/CD, applicazioni stateless, sviluppo locale, deployment cloud. VM: isolamento sicurezza critico, sistemi operativi diversi (Windows su Linux), workload con kernel custom. In produzione spesso si usano entrambi: VM come host sui cloud provider, container come workload.

1.4 L'ecosistema nel 2026

Dal monopolio di Docker si è passati a un ecosistema ricco di alternative e standard aperti.

Tabella 1.4: Ecosistema container nel 2026

Tool	Categoria	Caratteristiche
Docker Engine	Runtime + CLI	Il classico; 71% adoption (sviluppo)
containerd 2.0	Runtime OCI	Backend di Docker e Kubernetes
Podman 5.x	Runtime daemonless	Rootless, Docker-compatible, RHEL default
Buildah	Image builder	Daemonless, fine-grained, OCI
BuildKit	Build engine	Backend avanzato di <code>docker build</code>
Compose v5	Orchestrazione locale	Stack multi-container, Go SDK
Docker Scout	Security	Analisi CVE nelle immagini
Docker AI Catalog	AI/ML	Immagini ottimizzate per LLM

Docker nell'industria — dati 2025–2026

- **92%** delle aziende IT usa container Docker (Docker Survey 2025)
- **71%** degli sviluppatori usa Docker come tool (Stack Overflow Survey 2025, +17 punti percentuali YoY)
- In produzione: Docker cede terreno a containerd/CRI-O come runtime Kubernetes
- Compose v1 (`docker-compose` con il trattino) definitivamente rimosso ad aprile 2025

Fonti: Docker Survey 2025; Stack Overflow Developer Survey 2025; dev.to/dataformathub (2026)

1.5 Docker in cifre: adozione nel mondo reale

Tabella 1.5: Dati di adozione Docker 2025–2026

Metrica	Dato
Adozione in aziende IT	92% (Docker Survey 2025)
Sviluppatori che usano Docker	71% (Stack Overflow Survey 2025, +17 pp YoY)
Immagini su Docker Hub	Oltre 8 milioni
Pull mensili Docker Hub	Oltre 13 miliardi
Container attivi in produzione	Oltre 5 milioni (stime Datadog)
Riduzione tempo deployment	-40% medio con container vs bare metal
Riduzione costi infrastruttura	-20% medio per densità superiore

Fonti: Docker Survey 2025 (<https://www.docker.com/resources/>); Stack Overflow Developer Survey 2025 (<https://survey.stackoverflow.co/2025/>); Datadog State of Containers 2024 (<https://www.datadoghq.com/container-report/>).

1.6 Lo standard OCI: perché è importante

La creazione dell’OCI (**Open Container Initiative**) nel 2015 ha evitato la frammentazione dell’ecosistema container. Prima dell’OCI, ogni tool usava formati incompatibili.

Tabella 1.6: Standard OCI e loro impatto

Specifica	Versione	Cosa definisce
Runtime Spec	1.2	Come creare ed eseguire un container (runc)
Image Spec	1.1	Formato delle immagini e dei layer
Distribution Spec	1.1	Come le immagini vengono distribuite via registry

Grazie all’OCI, un’immagine costruita con Docker può essere: eseguita con Podman, archiviata su qualsiasi registry OCI-compliant (Docker Hub, GitHub Container Registry, Amazon ECR, Google AR), orchestrata da Kubernetes con qualsiasi CRI runtime.

Capitolo 2

Architettura Interna di Docker

2.1 Componenti del sistema Docker

Docker non è un monolite: è un insieme di componenti modulari che interagiscono tra loro seguendo gli standard OCI.

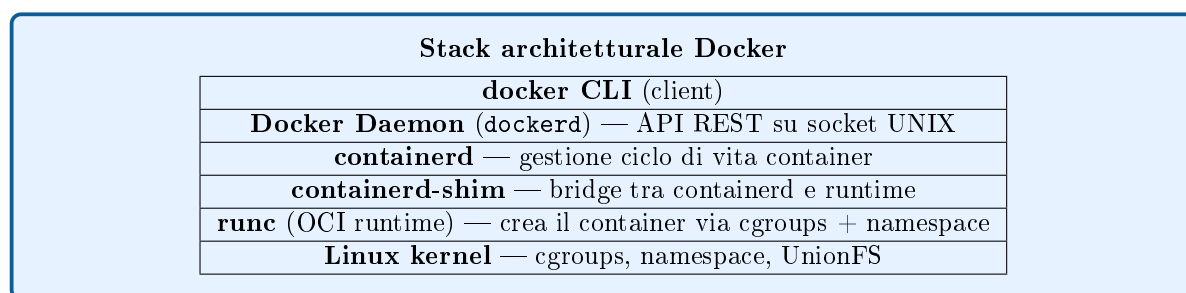


Figura 2.1: Architettura a strati di Docker Engine

Tabella 2.1: Componenti Docker e loro responsabilità

Componente	Linguaggio	Responsabilità
docker (CLI)	Go	Interfaccia utente; invia comandi via REST
dockerd	Go	Daemon principale; gestisce immagini, reti, volumi
containerd	Go	Runtime di alto livello; pull immagini, snapshotter
containerd-shim	Go	Mantiene il container vivo se containerd riavvia
runc	Go/C	OCI runtime; crea namespace, cgroups, avvia processo
BuildKit (buildkitd)	Go	Build engine avanzato con cache distribuita
registry v2	Go	Docker Hub / registry privato (OCI Distribution Spec)

2.2 Come funziona docker run

Quando si esegue `docker run ubuntu bash`, avviene una catena di operazioni ben precisa:

Listing 2.1: Tracciare il flusso di docker run

```
1 # Lancia un container e osserva i processi coinvolti
2 docker run -d --name test ubuntu sleep 3600 &
3
```

```
4 # Albero dei processi (richiede: apt install psmisc)
5 pstree -p | grep -A5 dockerd
6 # dockerd(1234) --- containerd(1235) --- containerd-shim(2000) --- sleep(2001)
7
8 # Socket API di dockerd
9 ls -la /var/run/docker.sock
10 # srw-rw---- 1 root docker /var/run/docker.sock
11
12 # Namespace del container
13 PID=$(docker inspect test --format '{{.State.Pid}}')
14 ls -la /proc/$PID/ns/
15 # net -> net:[4026532XXX] (isolato dall'host!)
16 # pid -> pid:[4026532YYY]
17
18 # cgroup del container
19 cat /proc/$PID/cgroup
20 # 0:/:system.slice/docker-<ID>.scope
21
22 # Verifica limiti cgroup v2
23 docker run --memory=256m --cpus=0.5 -d ubuntu sleep 60
24 cat /sys/fs/cgroup/system.slice/docker-*.scope/memory.max
25 # 268435456 (= 256 MB in byte)
```

2.3 UnionFS e i layer delle immagini

Le immagini Docker sono composte da **layer** immutabili sovrapposti. Il filesystem risultante è un'unione read-only dei layer + uno strato scrivibile in cima (per il container).

Listing 2.2: Ispezionare i layer di un'immagine

```
1 # Visualizza i layer di un'immagine
2 docker image history nginx:latest
3 # IMAGE          CREATED          CREATED BY          SIZE
4 # a7be6198544f   2 weeks ago     CMD ["nginx" "-g" "daemon\ldots{}" "0B
5 # <missing>      2 weeks ago     EXPOSE map[80/tcp:{}] 0B
6 # <missing>      2 weeks ago     COPY /docker-entrypoint.d\ldots{} 4.61kB
7 # <missing>      2 weeks ago     RUN /bin/sh -c apt-get ins\ldots{} 62.8MB
8 # <missing>      2 weeks ago     ENV NGINX_VERSION=1.25.4 0B
9 # <missing>      2 weeks ago     FROM debian:bookworm-slim 74.8MB
10
11 # Dettaglio layer in formato JSON
12 docker image inspect nginx:latest | python3 -m json.tool | \
13     grep -A5 'Layers'
14
15 # Dove sono fisicamente i layer? (storage driver overlay2)
16 docker info | grep 'Storage Driver'
17 # Storage Driver: overlay2
18
19 ls /var/lib/docker/overlay2/ | head -5
20 # Ogni directory = un layer dell'immagine
21
22 # Anatomia di un layer overlay2:
23 ls /var/lib/docker/overlay2/<ID>/
24 # diff/          <- contenuto del layer
25 # link           <- ID breve
26 # lower          <- puntatore ai layer sottostanti
```

```
27 # merged/      <- vista unificata (montata solo per container attivi)
28 # work/        <- area di lavoro overlay
```

Storage drivers disponibili

- **overlay2**: default e raccomandato (Linux kernel 4.0+ con ext4/xfs)
- **btrfs**: su Arch Linux con filesystem btrfs
- **zfs**: su sistemi con ZFS
- **devicemapper**: legacy, sconsigliato
- **vfs**: nessun copy-on-write, solo per test

2.4 Il registro OCI: Docker Hub e alternative

Le immagini vengono distribuite tramite **registry** conformi alla OCI Distribution Specification v1.

Listing 2.3: Interagire con registry pubblici e privati

```
1 # Docker Hub (default)
2 docker pull nginx:1.25-alpine
3 docker pull ubuntu:24.04
4
5 # Registry Google (gcr.io)
6 docker pull gcr.io/google-containers/pause:3.9
7
8 # Registry GitHub (ghcr.io)
9 docker pull ghcr.io/owner/repo:tag
10
11 # Avvia registry privato locale
12 docker run -d -p 5000:5000 --name registry \
13   --restart unless-stopped \
14   -v /opt/registry:/var/lib/registry \
15   registry:2
16
17 # Testa il registry locale
18 docker tag nginx:alpine localhost:5000/nginx:local
19 docker push localhost:5000/nginx:local
20 docker pull localhost:5000/nginx:local
21
22 # API v2 del registry (REST pura)
23 curl http://localhost:5000/v2/_catalog
24 # {'repositories': ['nginx']}
25
26 curl http://localhost:5000/v2/nginx/tags/list
27 # {'name': 'nginx', 'tags': ['local']}
```

2.5 BuildKit: il motore di build avanzato

Dal 2023 (Docker 23.0), **BuildKit** è il builder predefinito. Offre build parallele, cache avanzata, multi-platform e secrets sicuri.

Listing 2.4: BuildKit: funzionalità avanzate


```
1 # BuildKit e' gia' attivo di default in Docker 23+
2 # Verifica
3 docker build --help | grep buildkit
4 DOCKER_BUILDKIT=1 docker build . # forzarlo esplicitamente
5
6 # Cache di build da registry remoto
7 docker build \
8   --cache-from type=registry,ref=myregistry/myapp:cache \
9   --cache-to type=registry,ref=myregistry/myapp:cache,mode=max \
10  -t myregistry/myapp:latest .
11
12 # Build multi-platform (AMD64 + ARM64)
13 docker buildx create --use --name multi
14 docker buildx build \
15   --platform linux/amd64,linux/arm64 \
16   --push \
17   -t myregistry/myapp:latest .
18
19 # Secrets sicuri (non finiscono nei layer!)
20 docker build \
21   --secret id=mysecret,src=/run/secrets/mykey \
22   .
23 # Nel Dockerfile: RUN --mount=type=secret,id=mysecret ...
24
25 # Docker Bake: build dichiarativo (introdotto con Compose v5)
26 cat docker-bake.hcl
27 # group 'default' { targets = ['web', 'api'] }
28 # target 'web' { dockerfile = 'web/Dockerfile' ... }
29 docker buildx bake
```

Capitolo 3

Installazione di Docker

3.1 Installazione su Debian 12 (Bookworm) e Kali Linux

Listing 3.1: Installazione Docker Engine su Debian/Kali

```
1 # 1. Rimuovi versioni precedenti e conflitti
2 sudo apt remove -y docker docker-engine docker.io containerd runc \
3   docker-compose 2>/dev/null || true
4
5 # 2. Installa prerequisiti
6 sudo apt update
7 sudo apt install -y ca-certificates curl gnupg lsb-release
8
9 # 3. Aggiungi la chiave GPG ufficiale Docker
10 sudo install -m 0755 -d /etc/apt/keyrings
11 curl -fsSL https://download.docker.com/linux/debian/gpg | \
12   sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
13 sudo chmod a+r /etc/apt/keyrings/docker.gpg
14
15 # 4. Aggiungi il repository (auto-rileva Debian o Kali)
16 DISTRO=$(. /etc/os-release && echo "$ID")
17 [ "$DISTRO" = "kali" ] && DISTRO="debian" # Kali usa repo Debian
18
19 echo "deb [arch=$(dpkg --print-architecture) \
20   signed-by=/etc/apt/keyrings/docker.gpg] \
21   https://download.docker.com/linux/debian \
22   $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
23   sudo tee /etc/apt/sources.list.d/docker.list
24
25 # 5. Installa Docker Engine + Compose plugin
26 sudo apt update
27 sudo apt install -y \
28   docker-ce docker-ce-cli containerd.io \
29   docker-buildx-plugin docker-compose-plugin
30
31 # 6. Abilita e avvia il servizio
32 sudo systemctl enable docker --now
33
34 # 7. Aggiungi l'utente al gruppo docker (richiede logout/login)
35 sudo usermod -aG docker $USER
36 newgrp docker # applica subito senza logout
37
38 # 8. Verifica installazione
39 docker version
40 docker compose version
41 docker run --rm hello-world
```

3.2 Installazione su Arch Linux

Listing 3.2: Installazione Docker su Arch Linux

```
1 # Installa Docker e plugin Compose dal repository extra
2 sudo pacman -S docker docker-compose
3
4 # Abilita e avvia
5 sudo systemctl enable docker --now
6
7 # Aggiungi utente al gruppo
8 sudo usermod -aG docker $USER
9 newgrp docker
10
11 # Verifica
12 docker version
13 docker compose version
14
15 # Opzionale: Docker Desktop per Arch (via AUR)
16 # yay -S docker-desktop
17
18 # Per usare il backend btrfs (se il filesystem e' btrfs)
19 # /etc/docker/daemon.json:
20 # { 'storage-driver': 'btrfs' }
21 sudo systemctl restart docker
```

3.3 Configurazione post-installazione

Listing 3.3: Configurazione avanzata del daemon

```
1 # File di configurazione principale del daemon
2 sudo nano /etc/docker/daemon.json
3
4 # Esempio configurazione completa
5 cat << 'EOF' | sudo tee /etc/docker/daemon.json
6 {
7     'log-driver': 'json-file',
8     'log-opts': {
9         'max-size': '10m',
10        'max-file': '3'
11    },
12    'storage-driver': 'overlay2',
13    'default-address-pools': [
14        {'base': '172.17.0.0/16', 'size': 24}
15    ],
16    'live-restore': true,
17    'userland-proxy': false,
18    'no-new-privileges': true,
19    'features': {
20        'buildkit': true
21    }
22 }
23 EOF
24
25 sudo systemctl reload docker
```

```

26
27 # Verifica configurazione applicata
28 docker info | grep -E 'Storage|Logging|Live Restore'
29
30 # Proxy per ambienti aziendali
31 sudo mkdir -p /etc/systemd/system/docker.service.d/
32 cat << 'EOF' | sudo tee \
33     /etc/systemd/system/docker.service.d/http-proxy.conf
34 [Service]
35 Environment='HTTP_PROXY=http://proxy:3128'
36 Environment='HTTPS_PROXY=http://proxy:3128'
37 Environment='NO_PROXY=localhost,127.0.0.1,localregistry'
38 EOF
39 sudo systemctl daemon-reload && sudo systemctl restart docker

```

3.4 Docker senza root: modalità rootless

Docker in modalità **rootless** esegue il daemon senza privilegi root, aumentando significativamente la sicurezza in ambienti condivisi.

Listing 3.4: Configurazione Docker rootless

```

1 # Prerequisiti
2 sudo apt install -y uidmap slirp4netns
3
4 # Installa Docker rootless per l'utente corrente
5 dockerd-rootless-setuptool.sh install
6
7 # Aggiorna la variabile d'ambiente (aggiungi al ~/.bashrc)
8 export DOCKER_HOST=unix://$XDG_RUNTIME_DIR/docker.sock
9 export PATH=/usr/bin:$PATH
10
11 # Abilita come servizio utente (non root!)
12 systemctl --user enable docker
13 systemctl --user start docker
14
15 # Verifica: dockerd gira come utente normale
16 ps aux | grep dockerd
17 # mario 1234 ... dockerd (NON root!)
18
19 # Usa normalmente
20 docker run --rm hello-world

```

Attenzione

In modalità rootless alcune funzionalità non sono disponibili: port mapping < 1024, cgroups v1, e alcuni storage driver. Usare cgroups v2 (default su Debian 12 e Arch Linux) per la migliore compatibilità.

Capitolo 4

Dockerfile: Costruire Immagini

4.1 Istruzioni Dockerfile

Tabella 4.1: Istruzioni Dockerfile complete

Istruzione	Tipo	Descrizione
FROM	Base	Immagine base (prima istruzione obbligatoria)
RUN	Build	Esegue comando durante la build; crea layer
COPY	Filesystem	Copia file/dir dal contesto di build
ADD	Filesystem	Come COPY + URL remoti + auto-estrazione tar
WORKDIR	Ambiente	Imposta directory di lavoro
ENV	Ambiente	Variabile d'ambiente (persiste nel container)
ARG	Build	Variabile solo durante la build
EXPOSE	Rete	Documenta la porta (non la pubblica)
VOLUME	Storage	Punto di mount per dati persistenti
USER	Sicurezza	Cambia utente per comandi successivi
CMD	Runtime	Comando predefinito (sovrascrivibile)
ENTRYPOINT	Runtime	Comando fisso (CMD diventa argomento)
HEALTHCHECK	Monitor	Verifica salute del container
LABEL	Metadata	Coppie chiave=valore di metadati
SHELL	Build	Cambia la shell per RUN
STOPSIGNAL	Runtime	Segnale per fermare il container

4.2 Dockerfile annotato

Listing 4.1: Dockerfile Python Flask con best practice

```
1  # Specifica la versione esatta (mai usare :latest in produzione)
2  FROM python:3.12-slim
3
4  # Metadati OCI (pratiche moderne)
5  LABEL org.opencontainers.image.title='Mia App Flask' \
6        org.opencontainers.image.version='1.0.0' \
7        org.opencontainers.image.author='OTHack1A'
8
9  # ARG disponibile solo durante la build
10 ARG APP_ENV=production
11
12 # ENV disponibile anche a runtime
13 ENV PYTHONDONTWRITEBYTECODE=1 \
14     PYTHONUNBUFFERED=1 \
15     APP_ENV=${APP_ENV}
16
```

```
17 # Imposta directory di lavoro
18 WORKDIR /app
19
20 # Prima COPY e installa dipendenze (layer cacheabile!)
21 # Se solo requirements.txt cambia, questo layer viene invalidato
22 # Solo se requirements.txt cambia, il layer viene invalidato
23 COPY requirements.txt .
24 RUN pip install --no-cache-dir -r requirements.txt
25
26 # Poi copia il sorgente (cambia piu' spesso -> layer separato)
27 COPY app/ .
28
29 # Crea utente non-root per sicurezza
30 RUN useradd --create-home --shell /bin/bash appuser \
31     && chown -R appuser:appuser /app
32 USER appuser
33
34 # Documenta la porta (non la espone: serve -p al run)
35 EXPOSE 5000
36
37 # Healthcheck: il container viene marcato unhealthy se fallisce
38 HEALTHCHECK --interval=30s --timeout=5s \
39     --start-period=10s --retries=3 \
40     CMD python -c 'import urllib.request; \
41         urllib.request.urlopen('http://localhost:5000/health')' \
42         || exit 1
43
44 # CMD: comando predefinito, sovrascrivibile con docker run ... CMD
45 CMD ['gunicorn', '--bind', '0.0.0.0:5000', \
46     '--workers', '2', 'app:app']
```

4.3 Multi-stage build

La tecnica **multi-stage** separa il build environment dal runtime, producendo immagini finali molto più piccole.

Listing 4.2: Multi-stage build: Go application

```
1 # -- Stage 1: build --
2 FROM golang:1.22-alpine AS builder
3
4 # Installa solo le dipendenze di build
5 RUN apk add --no-cache git ca-certificates tzdata
6
7 WORKDIR /src
8
9 # Copia prima go.mod e go.sum per sfruttare la cache
10 COPY go.mod go.sum ./
11 RUN go mod download
12
13 # Compila con ottimizzazioni per ridurre la dimensione
14 COPY . .
15 RUN CGO_ENABLED=0 GOOS=linux GOARCH=amd64 \
16     go build \
17     -ldflags='-w -s -X main.Version=$(git describe --tags)' \
18     -o /bin/server ./cmd/server
```

```

19
20 # -- Stage 2: runtime minimale --
21 # scratch = immagine vuota (0 MB di base)
22 FROM scratch
23
24 # Copia solo il necessario dal builder
25 COPY --from=builder /etc/ssl/certs/ca-certificates.crt \
26     /etc/ssl/certs/
27 COPY --from=builder /usr/share/zoneinfo /usr/share/zoneinfo
28 COPY --from=builder /bin/server /server
29
30 EXPOSE 8080
31 ENTRYPOINT ["/server"]
32
33 # Risultato: immagine di pochi MB invece di 300+ MB con Go base

```

Listing 4.3: Confronto dimensioni immagine

```

1 # Costruisci le due versioni e confronta
2 docker build -f Dockerfile.single -t app:single .
3 docker build -f Dockerfile.multi -t app:multi .
4
5 docker image ls | grep app
6 # app single abc123 ... 310MB
7 # app multi def456 ... 8MB <- 40x piu' piccola!
8
9 # Analisi layer con dive (tool di terze parti)
10 # sudo apt install dive oppure yay -S dive
11 dive app:single

```

4.4 .dockerignore

Come `.gitignore`, il file `.dockerignore` esclude file dal contesto di build, riducendo il tempo di trasferimento e la dimensione del contesto.

Listing 4.4: `.dockerignore` tipico

```

1 # Controllo versione
2 .git
3 .gitignore
4 .github/
5
6 # Documentazione
7 README.md
8 docs/
9 *.md
10
11 # Ambienti virtuali e dipendenze locali
12 node_modules/
13 __pycache__/
14 *.pyc
15 *.pyo
16 .venv/
17 venv/
18
19 # Test e CI

```

```
20 tests/
21 .pytest_cache/
22 coverage.xml
23 .coverage
24
25 # Artefatti di build locali
26 dist/
27 build/
28 *.egg-info/
29
30 # Configurazioni editor
31 .vscode/
32 .idea/
33 *.swp
34
35 # Segreti (MAI nel contesto di build!)
36 .env
37 .env.local
38 *.pem
39 *.key
40 secrets/
41
42 # Log
43 *.log
44 logs/
```

4.5 Best practice Dockerfile

Listing 4.5: Pattern da EVITARE vs pattern corretti

```
1 # -- EVITARE: latest e comandi non deterministici --
2 FROM ubuntu:latest # NON fare: versione imprevedibile
3 RUN apt-get update # NON fare: in layer separato invalida cache
4 RUN apt-get install -y curl # ogni build puo' installare versioni diverse
5
6 # -- CORRETTO --
7 FROM ubuntu:24.04 # versione fissa
8 RUN apt-get update && apt-get install -y \
9     curl=8.5.0-2ubuntu10 \ # versione fissa se critico
10    && rm -rf /var/lib/apt/lists/* # pulisci in STESSO layer
11
12 # -- EVITARE: ADD quando si vuole solo copiare --
13 ADD app.tar.gz /app/ # OK solo se serve auto-estrazione
14
15 # -- CORRETTO: COPY per file semplici --
16 COPY app/ /app/
17
18 # -- EVITARE: eseguire come root --
19 # (nessun USER -> implicitamente root)
20 CMD ['python', 'app.py']
21
22 # -- CORRETTO: utente non-root --
23 RUN useradd -m appuser
24 USER appuser
25 CMD ['python', 'app.py']
26
```



```
27 # -- EVITARE: segreti come ARG o ENV --
28 ARG DB_PASSWORD=secret123 # visibile in docker history!
29 ENV API_KEY=abc123         # visibile in docker inspect!
30
31 # -- CORRETTO: secrets BuildKit --
32 # docker build --secret id=api_key,src=.api_key .
33 RUN --mount=type=secret,id=api_key \
34     API_KEY=$(cat /run/secrets/api_key) \
35     && python setup.py configure
```

4.6 Ottimizzazione avanzata dei layer

4.6.1 L'ordine degli strati conta

La cache di BuildKit invalida un layer e tutti i layer successivi quando un layer cambia. Capire questo meccanismo permette di costruire Dockerfile che ricompilano solo ciò che è cambiato.

Listing 4.6: Ordinamento ottimale dei layer per cache massima

```
1 FROM python:3.12-slim
2
3 WORKDIR /app
4
5 # -- LAYER 1: Sistema operativo (cambia raramente) --
6 # Installa dipendenze di sistema una volta sola.
7 # Queste cambiano solo quando aggiorni le versioni.
8 RUN apt-get update && apt-get install -y --no-install-recommends \
9     libpq-dev \
10     gcc \
11     && apt-get clean \
12     && rm -rf /var/lib/apt/lists/*
13
14 # -- LAYER 2: Dipendenze Python (cambia raramente) --
15 # Copia SOLO requirements.txt. Se l'app cambia ma le dipendenze
16 # no, questo layer rimane in cache!
17 COPY requirements.txt .
18
19 # Usa cache mount: la cache di pip persiste tra build diverse
20 RUN --mount=type=cache,target=/root/.cache/pip \
21     pip install --no-cache-dir -r requirements.txt
22
23 # -- LAYER 3: Configurazione (cambia ogni tanto) --
24 COPY config/ ./config/
25
26 # -- LAYER 4: Sorgente applicazione (cambia spesso) --
27 # Questo e' l'ultimo layer: invalida solo se stesso, non
28 # le dipendenze che stanno sopra nella pila.
29 COPY app/ ./app/
30
31 # -- LAYER 5: Metadata (rebuild velocissimo) --
32 USER nobody
33 EXPOSE 5000
34 CMD ["gunicorn", "app.main:app"]
```

4.6.2 Analisi visuale dei layer con dive

Listing 4.7: Analisi layer con dive

```
1 # Installa dive
2 sudo apt install dive          # Debian/Kali
3 # oppure:
4 curl -LO https://github.com/wagoodman/dive/releases/latest/\
5 download/dive_linux_amd64.tar.gz
6 tar -xzf dive_linux_amd64.tar.gz && sudo mv dive /usr/local/bin/
7
8 # Analisi interattiva (TUI)
9 dive myapp:latest
10
11 # Controlli: frecce per navigare layer, Tab per passare tra pannelli
12 # Mostra: layer -> file modificati -> efficienza cache
13
14 # Analisi non-interattiva (CI/CD)
15 dive --ci myapp:latest
16 # Exit code 1 se efficienza < 90%
17
18 # Imposta soglia personalizzata
19 dive --ci \
20   --ci-config .dive-ci.yaml \
21   myapp:latest
22 # .dive-ci.yaml:
23 # rules:
24 #   lowestEfficiency: 0.95
25 #   highestWastedBytes: '20MB'
26 #   highestUserWastedPercent: 0.10
```

4.7 Dockerfile per linguaggi specifici

Listing 4.8: Dockerfile ottimizzato per Node.js

```
1 FROM node:20-alpine AS base
2
3 # Installa dipendenze di sistema
4 RUN apk add --no-cache libc6-compat
5
6 WORKDIR /app
7
8 # -- Stage dipendenze: solo package.json --
9 FROM base AS deps
10 COPY package.json package-lock.json ./
11 # npm ci: installa esattamente package-lock.json (riproducibile)
12 RUN --mount=type=cache,target=/root/.npm \
13     npm ci --omit=dev
14
15 # -- Stage build --
16 FROM base AS builder
17 COPY --from=deps /app/node_modules ./node_modules
18 COPY . .
19 RUN npm run build      # TypeScript compile, webpack, ecc.
20
21 # -- Stage runtime: solo il necessario --
```

```
22 FROM base AS runner
23 ENV NODE_ENV=production
24
25 # Utente non-root
26 RUN addgroup --system --gid 1001 nodejs
27 RUN adduser --system --uid 1001 nextjs
28
29 # Copia SOLO gli artefatti della build
30 COPY --from=builder --chown=nextjs:nodejs /app/.next ./next
31 COPY --from=builder /app/node_modules ./node_modules
32 COPY --from=builder /app/package.json ./
33
34 USER nextjs
35 EXPOSE 3000
36 CMD [ 'node', 'server.js' ]
37
38 # Risultato:
39 # Base image + node_modules prod: ~180 MB
40 # Rispetto a: dev deps + src: ~1.2 GB
```

Capitolo 5

Docker Compose: Stack Multi-Container

5.1 Evoluzione di Compose

Tabella 5.1: Evoluzione di Docker Compose

Anno	Versione	Cambiamenti chiave
2013	fig	Precursore open source
2015	Compose v1	Acquisito da Docker; <code>docker-compose</code> in Python
2020	Compose v2	Riscritto in Go; plugin CLI (<code>docker compose</code>)
2022	Spec standard	Compose Specification: formato rolling senza version:
Apr 2025	Compose v1 EOL	<code>docker-compose</code> (con trattino) rimosso
Dic 2025	Compose Spec v5	“Mont Blanc”; Docker Bake integrato
2025	Compose v5	Go SDK ufficiale; stessa funzionalità di v2

Attenzione

Dal **aprile 2025** il comando `docker-compose` (con il trattino, versione Python v1) è stato rimosso definitivamente. Usare sempre `docker compose` (senza trattino, plugin Go v2/v5). Rimuovere anche il campo `version:` all’inizio del `compose.yaml`: è ignorato da Compose v2+ e ufficialmente deprecato.

5.2 Struttura di `compose.yaml`

Listing 5.1: `compose.yaml` — struttura completa commentata

```
1 # compose.yaml (nome preferito; docker-compose.yml ancora valido)
2 # NON scrivere: version: '3.8' <- ignorato e deprecato
3
4 services:
5
6   # -- Servizio web (applicazione) --
7   web:
8     # Build dall'immagine locale
9     build:
10       context: .           # directory del Dockerfile
11       dockerfile: Dockerfile # nome esplicito
12       args:                # ARG del Dockerfile
13         APP_ENV: production
14       cache_from:
15         - type=registry,ref=myregistry/web:cache
16
17     image: myregistry/web:latest # tag da assegnare
18
```

```
19   ports:
20     - '8080:80'                                     # host:container
21     - '127.0.0.1:8443:443'                           # bind solo su loopback
22
23   environment:
24     DATABASE_URL: postgresql://user:pass@db:5432/myapp
25     REDIS_URL: redis://cache:6379
26     LOG_LEVEL: info
27
28   # Leggi da file .env (NON committare in git con segreti!)
29   env_file:
30     - .env.production
31
32   depends_on:
33     db:
34       condition: service_healthy # attende healthcheck OK
35     cache:
36       condition: service_started
37
38   volumes:
39     - ./html:/var/www/html:ro # :ro = sola lettura
40     - uploads:/app/uploads    # volume named
41
42   networks:
43     - frontend
44     - backend
45
46   restart: unless-stopped # riavvia sempre tranne stop manuale
47
48   deploy:
49     resources:
50       limits:
51         cpus: '0.5'
52         memory: 512M
53       reservations:
54         memory: 256M
55
56   healthcheck:
57     test: ['CMD', 'curl', '-f', 'http://localhost/health']
58     interval: 30s
59     timeout: 5s
60     retries: 3
61     start_period: 10s
62
63   logging:
64     driver: json-file
65     options:
66       max-size: '10m'
67       max-file: '3'
68
69   # -- Database PostgreSQL --
70   db:
71     image: postgres:16-alpine
72     environment:
73       POSTGRES_DB: myapp
74       POSTGRES_USER: user
75       POSTGRES_PASSWORD: pass # in produzione: usa secrets!
```

```
76     volumes:
77         - db_data:/var/lib/postgresql/data
78         - ./init.sql:/docker-entrypoint-initdb.d/init.sql:ro
79     networks:
80         - backend
81     healthcheck:
82         test: ['CMD-SHELL', 'pg_isready -U user -d myapp']
83         interval: 10s
84         timeout: 5s
85         retries: 5
86         start_period: 30s
87
88     # -- Cache Redis --
89     cache:
90         image: redis:7-alpine
91         command: >
92             redis-server
93             --appendonly yes
94             --maxmemory 256mb
95             --maxmemory-policy allkeys-lru
96         volumes:
97             - redis_data:/data
98         networks:
99             - backend
100
101     # -- Reverse proxy Nginx --
102     proxy:
103         image: nginx:1.25-alpine
104         ports:
105             - '80:80'
106             - '443:443'
107         volumes:
108             - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
109             - ./certs:/etc/nginx/certs:ro
110         depends_on:
111             - web
112         networks:
113             - frontend
114         restart: unless-stopped
115
116     # -- Volumi named --
117     volumes:
118         db_data:
119             driver: local
120         redis_data:
121             driver: local
122         uploads:
123             driver: local
124
125     # -- Reti --
126     networks:
127         frontend:
128             driver: bridge
129         backend:
130             driver: bridge
131             internal: true # non accessibile dall'esterno host
```

5.3 Comandi Compose essenziali

Listing 5.2: Ciclo di vita completo con docker compose

```
1 # -- AVVIO --
2 # Avvia in background (-d = detached)
3 docker compose up -d
4
5 # Avvia con build forzata
6 docker compose up -d --build
7
8 # Avvia solo alcuni servizi
9 docker compose up -d web db
10
11 # Scala un servizio a 3 istanze
12 docker compose up -d --scale web=3
13
14 # -- STATO E LOG --
15 # Stato di tutti i container
16 docker compose ps
17
18 # Log in tempo reale (tutti i servizi)
19 docker compose logs -f
20
21 # Log di un servizio specifico
22 docker compose logs -f web
23
24 # Ultime 100 righe di log del DB
25 docker compose logs --tail=100 db
26
27 # -- OPERAZIONI --
28 # Riavvia solo un servizio
29 docker compose restart web
30
31 # Aggiorna un'immagine e ricrea il container
32 docker compose pull web && docker compose up -d web
33
34 # Esegui comando in container attivo
35 docker compose exec web bash
36 docker compose exec db psql -U user -d myapp
37
38 # Esegui comando in nuovo container (usa immagine esistente)
39 docker compose run --rm web python manage.py migrate
40
41 # -- ARRESTO E PULIZIA --
42 # Ferma senza rimuovere
43 docker compose stop
44
45 # Ferma e rimuove container + reti
46 docker compose down
47
48 # Ferma, rimuove container, reti E volumi (ATTENZIONE: distrugge i dati!)
49 docker compose down -v
50
51 # Rimuove anche le immagini buildate localmente
52 docker compose down --rmi local
53
```

```

54 # -- FILE MULTIPLI (override) --
55 # Sviluppo: aggiunge debug ports, volume hot-reload
56 docker compose -f compose.yaml -f compose.dev.yaml up -d
57
58 # Produzione
59 docker compose -f compose.yaml -f compose.prod.yaml up -d

```

5.4 Watch mode: sviluppo con hot-reload

Compose v2 introduce la **watch mode** per il reload automatico durante lo sviluppo:

Listing 5.3: Watch mode in compose.yaml (sviluppo)

```

1 services:
2   web:
3     build: .
4     develop:
5       watch:
6         # Sincronizza i sorgenti Python senza rebuild
7         - action: sync
8           path: ./app
9           target: /app
10        ignore:
11          - __pycache__/_
12          - '*.pyc'
13        # Rebuilda se requirements.txt cambia
14        - action: rebuild
15          path: requirements.txt
16        # Ricrea il container se docker-compose.yml cambia
17        - action: sync+restart
18          path: ./config
19          target: /app/config

```

Listing 5.4: Avviare la watch mode

```

1 # Avvia con watch mode attiva (hot-reload)
2 docker compose watch
3
4 # Oppure con up
5 docker compose up --watch

```

5.5 Profiles: configurazioni condizionali

Listing 5.5: Profiles in compose.yaml: servizi opzionali

```

1 # I profiles permettono di attivare servizi selettivamente
2 # senza file compose separati
3 services:
4
5   # Sempre attivo (nessun profile)
6   web:
7     image: nginx:alpine
8     ports: ['80:80']
9

```



```
10  # Solo in sviluppo (profile: debug)
11  mailhog:
12      image: mailhog/mailhog
13      profiles: [debug]
14      ports: ['8025:8025']      # web UI per email in dev
15
16  # Solo per test (profile: testing)
17  selenium:
18      image: selenium/standalone-chrome
19      profiles: [testing]
20      shm_size: 2gb
21
22  # Solo per monitoring (profile: monitoring)
23  prometheus:
24      image: prom/prometheus
25      profiles: [monitoring]
26      ports: ['9090:9090']
27
28  grafana:
29      image: grafana/grafana
30      profiles: [monitoring]
31      ports: ['3000:3000']
32
33  # Attivo in piu' profile
34  debug-tools:
35      image: nicolaka/netshoot
36      profiles: [debug, testing]
37      command: sleep infinity
```

Listing 5.6: Usare i profiles in compose

```
1  # Avvia solo servizi base (senza profile)
2  docker compose up -d
3
4  # Avvia con profile specifico
5  docker compose --profile debug up -d
6  # Avvia: web + mailhog + debug-tools
7
8  # Piu' profile contemporaneamente
9  docker compose --profile debug --profile monitoring up -d
10 # Avvia: web + mailhog + debug-tools + prometheus + grafana
11
12 # Variabile d'ambiente (alternativa)
13 COMPOSE_PROFILES=debug,monitoring docker compose up -d
14
15 # Lista servizi di un profile
16 docker compose --profile testing config --services
```

5.6 Variabili d'ambiente e interpolazione

Listing 5.7: Interpolazione variabili in compose.yaml

```
1  # compose.yaml supporta interpolazione di variabili
2  # con sintassi ${VARIABILE:-default} e ${VARIABILE:?errore}
3
```

```

4  # -- File .env (caricato automaticamente) --
5  cat > .env << 'EOF'
6  # Configurazione ambiente di default (sviluppo)
7  APP_VERSION=1.0.0-dev
8  DATABASE_HOST=db
9  DATABASE_PORT=5432
10 POSTGRES_PASSWORD=devsecret
11 REDIS_MAXMEM=256mb
12 LOG_LEVEL=debug
13 EOF
14
15 # compose.yaml che usa le variabili:
16 # services:
17 #   web:
18 #     image: myapp:${APP_VERSION:-latest}
19 #     environment:
20 #       DB_URL: postgresql://user:${POSTGRES_PASSWORD}@${DATABASE_HOST}:${
21 #         DATABASE_PORT}/myapp
22 #       LOG_LEVEL: ${LOG_LEVEL:-info}
23 #
24 #   redis:
25 #     image: redis:alpine
26 #     command: redis-server --maxmemory ${REDIS_MAXMEM:?REDIS_MAXMEM
27 #       richiesta}
28 #     # :? = errore se non definita
29
30 # -- File .env per ambienti diversi --
31 docker compose --env-file .env.staging up -d
32 docker compose --env-file .env.production up -d
33
34 # -- Variabili con valori default --
35 # ${VAR:-default}      = usa default se VAR non definita o vuota
36 # ${VAR-default}      = usa default solo se VAR non definita
37 # ${VAR:?messaggio}    = errore se VAR non definita o vuota
38 # ${VAR?messaggio}    = errore solo se VAR non definita
39
40 # Verifica le variabili risolte
41 docker compose config # mostra la config con variabili sostituite

```

Capitolo 6

Networking e Volumi

6.1 Reti Docker

Tabella 6.1: Driver di rete Docker

Driver	Scope	Caratteristiche
bridge	Singolo host	Default; rete privata con NAT verso host
host	Singolo host	Usa la rete dell'host direttamente (no isolamento)
overlay	Multi-host	Swarm mode; tunnel VXLAN tra nodi
macvlan	Singolo host	Container con MAC address proprio (come bare metal)
ipvlan	Singolo host	Come macvlan ma con IP sharing
none	—	Nessuna rete; isolamento totale

Listing 6.1: Gestione reti Docker

```
1 # Crea rete bridge personalizzata
2 docker network create \
3   --driver bridge \
4   --subnet 192.168.100.0/24 \
5   --gateway 192.168.100.1 \
6   --opt com.docker.network.bridge.name=docker_mynet \
7   mynet
8
9 # Lista reti
10 docker network ls
11 docker network inspect mynet
12
13 # Container nella stessa rete si raggiungono per nome
14 docker run -d --name db --network mynet postgres:16-alpine
15 docker run -d --name web --network mynet nginx
16 # web puo' raggiungere db con: http://db:5432
17
18 # Connetti un container a piu' reti
19 docker network connect mynet existing_container
20 docker network disconnect mynet existing_container
21
22 # Diagnostica rete
23 docker exec web ping -c3 db
24 docker exec web nslookup db # Docker ha un DNS interno!
25
26 # Rete host (solo Linux, non Mac/Windows)
27 docker run --rm --network host nginx
28 # nginx ascolta sulla porta 80 dell'host direttamente
29
30 # Rete macvlan (container come dispositivo di rete fisico)
```

```
31 docker network create \  
32 --driver macvlan \  
33 --subnet 192.168.1.0/24 \  
34 --gateway 192.168.1.1 \  
35 -o parent=eth0 \  
36 macvlan_net
```

6.2 Volumi e persistenza dei dati

I **volumi** Docker sono il meccanismo raccomandato per i dati persistenti. A differenza dei bind mount, sono gestiti completamente da Docker.

Tabella 6.2: Tipi di mount in Docker

Tipo	Sintassi	Uso tipico
Volume named	-v myvolume:/data	Database, dati app
Bind mount	-v /host/path:/data	Sviluppo (hot-reload)
tmpfs	-tmpfs /tmp	Dati temporanei in RAM
Volume anonimo	-v /data	Dati temporanei (sconsigliato)

Listing 6.2: Gestione volumi

```
1 # Crea volume named  
2 docker volume create db_data  
3  
4 # Lista volumi  
5 docker volume ls  
6 docker volume inspect db_data  
7 # Mostra dove Docker ha salvato fisicamente i dati:  
8 # /var/lib/docker/volumes/db_data/_data  
9  
10 # Usa il volume  
11 docker run -d \  
12 --name postgres \  
13 -v db_data:/var/lib/postgresql/data \  
14 -e POSTGRES_PASSWORD=secret \  
15 postgres:16-alpine  
16  
17 # Backup di un volume  
18 docker run --rm \  
19 -v db_data:/source:ro \  
20 -v $(pwd):/backup \  
21 alpine \  
22 tar czf /backup/db_backup_$(date +%Y%m%d).tar.gz -C /source .  
23  
24 # Restore di un volume  
25 docker run --rm \  
26 -v db_data:/target \  
27 -v $(pwd):/backup:ro \  
28 alpine \  
29 tar xzf /backup/db_backup_20260613.tar.gz -C /target  
30  
31 # Rimuovi volumi non usati (ATTENZIONE: irreversibile)
```

```
32 docker volume prune
33 docker volume rm db_data
34
35 # Bind mount per sviluppo (hot-reload)
36 docker run -d \
37   --name webapp \
38   -v $(pwd)/src:/app/src \ # modifica src -> visibile nel container
39   -p 8000:8000 \
40   myapp:dev
```

6.3 Docker DNS e service discovery

I container nella stessa rete Docker si scoprono automaticamente tramite il **DNS interno** di Docker (resolver embedded in dockerd).

Listing 6.3: DNS Docker: service discovery automatico

```
1 # Docker gestisce un resolver DNS su 127.0.0.11 (in ogni container)
2 docker exec mycontainer cat /etc/resolv.conf
3 # nameserver 127.0.0.11
4 # options ndots:0
5
6 # I container si raggiungono per nome del servizio
7 # In compose.yaml, 'web' puo' raggiungere 'db' come hostname
8 docker compose exec web curl http://api:8000/health
9
10 # Alias di rete (stesso container, nomi multipli)
11 docker network connect \
12   --alias api \
13   --alias backend \
14   mynet mycontainer
15
16 # Risoluzione: supporta anche task.nome_servizio per Swarm
17 # es: task.web.1, task.web.2...
18
19 # Testa la risoluzione DNS
20 docker exec web nslookup db
21 # Server: 127.0.0.11
22 # Address: 127.0.0.11:53
23 # Name: db
24 # Address: 172.20.0.3
```

6.4 Port publishing e forwarding

Listing 6.4: Opzioni di port publishing

```
1 # Pubblica porta 80 del container sulla 8080 dell'host
2 docker run -p 8080:80 nginx
3
4 # Pubblica su interfaccia specifica (sicurezza!)
5 docker run -p 127.0.0.1:8080:80 nginx # solo localhost
6
7 # Pubblica UDP
8 docker run -p 53:53/udp dns_server
```

```
9
10 # Pubblica range di porte
11 docker run -p 8080-8090:8080-8090 myapp
12
13 # Porta host casuale (utile per test)
14 docker run -P nginx # Docker sceglie la porta host
15 docker port nginx # mostra il mapping scelto
16 # 80/tcp -> 0.0.0.0:32768
17
18 # In compose.yaml: la sintassi estesa e' piu' esplicita
19 # ports:
20 #   - target: 80
21 #     published: 8080
22 #     protocol: tcp
23 #     mode: host
```

6.5 Reverse proxy con Traefik

Listing 6.5: Traefik come reverse proxy automatico

```
1 # Traefik scopre automaticamente i container Docker
2 # e configura il routing tramite labels
3 services:
4
5 # -- Traefik: il reverse proxy --
6 traefik:
7   image: traefik:v3.1
8   command:
9     - '--api.insecure=true' # dashboard su :8080
10    - '--providers.docker=true' # legge label Docker
11    - '--providers.docker.exposedbydefault=false'
12    - '--entrypoints.web.address=:80'
13    - '--entrypoints.websecure.address=:443'
14    # Let's Encrypt HTTPS automatico
15    - '--certificatesresolvers.letsencrypt.acme.email=mail@esempio.it'
16    - '--certificatesresolvers.letsencrypt.acme.storage=/certs/acme.json'
17    - '--certificatesresolvers.letsencrypt.acme.httpchallenge.entrypoint=web'
18   ports:
19     - '80:80'
20     - '443:443'
21     - '8080:8080' # Traefik Dashboard
22   volumes:
23     - /var/run/docker.sock:/var/run/docker.sock:ro
24     - certs:/certs
25   networks:
26     - proxy
27
28 # -- App A: routing per hostname --
29 app_a:
30   image: myapp-a:latest
31   labels:
32     - 'traefik.enable=true'
33     - 'traefik.http.routers.app-a.rule=Host(`app-a.esempio.it`)'
34     - 'traefik.http.routers.app-a.entrypoints=websecure'
```

```
35     - 'traefik.http.routers.app-a.tls.certresolver=letsencrypt'
36     - 'traefik.http.services.app-a.loadbalancer.server.port=3000'
37     networks:
38     - proxy
39     - internal
40
41     # -- App B: routing per path prefix --
42     app_b:
43     image: myapp-b:latest
44     labels:
45     - 'traefik.enable=true'
46     - 'traefik.http.routers.app-b.rule=Host('esempio.it') && PathPrefix
47       ('/api')'
47     - 'traefik.http.routers.app-b.entrypoints=websecure'
48     - 'traefik.http.middlewares.strip-api.strip.prefixes=/api'
49     - 'traefik.http.routers.app-b.middlewares=strip-api'
50     networks:
51     - proxy
52     - internal
53
54     volumes:
55     certs:
56
57     networks:
58     proxy:
59     external: true      # crea con: docker network create proxy
60     internal:
61     internal: true
```

6.6 Volumi avanzati: backup e restore

Listing 6.6: Strategie di backup per volumi Docker

```
1  # -- Backup con container temporaneo --
2  # Pattern: monta il volume + una directory host, comprimi
3  docker run --rm \
4  -v postgres_data:/data:ro \
5  -v $(pwd)/backups:/backup \
6  alpine \
7  tar czf /backup/postgres_$(date +%Y%m%d_%H%M%S).tar.gz -C /data .
8
9  # -- Backup database PostgreSQL (hot backup) --
10 docker exec postgres \
11 pg_dump -U postgres mydb | \
12 gzip > ./backups/mydb_$(date +%Y%m%d).sql.gz
13
14 # Restore PostgreSQL
15 gunzip -c ./backups/mydb_20260613.sql.gz | \
16 docker exec -i postgres psql -U postgres mydb
17
18 # -- Replica volume su server remoto --
19 # Con rsync via SSH (incrementale)
20 docker run --rm \
21 -v mydata:/source:ro \
22 -v ~/.ssh:/root/.ssh:ro \
23 alpine sh -c 'apk add rsync openssh-client && \
```

```
24     rsync -avz /source/ user@backup-server:/backups/mydata/‘‘
25
26 # -- Snapshot LVM per backup istantaneo --
27 # Se /var/lib/docker e' su LVM:
28 sudo lvcreate -L10G -s -n docker_snap /dev/vg0/docker
29 sudo mount /dev/vg0/docker_snap /mnt/snap
30 tar czf /backup/docker_$(date +%Y%m%d).tar.gz /mnt/snap/volumes/
31 sudo umount /mnt/snap
32 sudo lvremove -f /dev/vg0/docker_snap
```


Capitolo 7

Sicurezza in Docker

7.1 Il modello di sicurezza di Docker

Docker non è una sandbox sicura di default: condivide il kernel con l'host. Un container compromesso con escape exploit può compromettere l'intero sistema.

Listing 7.1: Analisi della superficie di attacco

```
1 # Verifica se un container gira come root
2 docker inspect mycontainer | python3 -m json.tool | \
3     grep -A3 'User'
4 # 'User': '' <- stringa vuota = root (PERICOLOSO!)
5
6 # Capability del container di default
7 docker run --rm ubuntu capsh --print | grep Current
8 # Current: cap_chown,cap_dac_override,...,cap_net_raw,...
9
10 # Con --cap-drop=ALL si rimuovono tutte le capability
11 docker run --rm --cap-drop=ALL ubuntu capsh --print
12 # Current: =
13
14 # Verifica seccomp profile attivo
15 docker inspect mycontainer | grep -A5 SeccompProfile
16
17 # Verifica AppArmor profile
18 docker inspect mycontainer | grep AppArmorProfile
```

7.2 Hardening dei container

Listing 7.2: Opzioni di sicurezza al runtime

```
1 # Container con tutte le capability rimosse
2 # e solo quelle necessarie riabilitate
3 docker run \
4     --cap-drop=ALL \
5     --cap-add=NET_BIND_SERVICE \ # per porta < 1024
6     --cap-add=CHOWN \           # se necessario
7     --read-only \               # filesystem read-only!
8     --tmpfs /tmp:rw,size=100m \ # /tmp scrivibile in RAM
9     --security-opt no-new-privileges:true \ # no setuid escalation
10    --security-opt seccomp=seccomp.json \   # profilo seccomp custom
11    --user 1000:1000 \                     # utente non-root
12    --memory=512m \                         # limite RAM
13    --cpus=0.5 \                           # limite CPU
14    myapp:latest
15
16 # In compose.yaml:
```

```
17 # security_opt:
18 #   - no-new-privileges:true
19 #   - seccomp:seccomp.json
20 # cap_drop: [ALL]
21 # cap_add: [NET_BIND_SERVICE]
22 # read_only: true
```

7.3 Docker Scout: analisi CVE nelle immagini

Listing 7.3: Docker Scout: vulnerability scanning

```
1 # Analisi vulnerabilita' di un'immagine
2 docker scout cves nginx:latest
3
4 # Report con fix disponibili
5 docker scout cves --format markdown nginx:latest > report.md
6
7 # Confronta due versioni
8 docker scout compare \
9   --to nginx:1.24-alpine \
10  nginx:1.25-alpine
11
12 # Quick summary
13 docker scout quickview nginx:latest
14 # Image: nginx:latest
15 # Digest: sha256:abc123...
16 # Vulnerabilities: 12 C / 45 H / 103 M / 67 L
17 # Packages: 89 (67 with known vulnerabilities)
18
19 # Raccomandazioni di base immagine
20 docker scout recommendations nginx:latest
```

7.4 Secrets management

Listing 7.4: Gestione sicura dei segreti

```
1 # -- BuildKit secrets (build time) --
2 # Crea file segreto
3 echo 'mia_api_key_super_segreta' > /tmp/api_key
4
5 # Build con secret (NON entra nei layer!)
6 docker build \
7   --secret id=api_key,src=/tmp/api_key \
8   -t myapp:latest .
9
10 # Nel Dockerfile:
11 # RUN --mount=type=secret,id=api_key \
12 #   export API_KEY=$(cat /run/secrets/api_key) && ...
13
14 # -- Docker Swarm secrets (runtime) --
15 # Crea un secret in Swarm
16 echo 'password_db_sicura' | docker secret create db_password -
17 docker secret ls
18
```

```
19 # Usa il secret nel servizio
20 docker service create \
21   --secret db_password \
22   --env DB_PASSWORD_FILE=/run/secrets/db_password \
23   myapp:latest
24
25 # -- File .env per sviluppo (NON per produzione!) --
26 cat .env
27 # DATABASE_URL=postgresql://user:pass@localhost:5432/dev
28 # API_KEY=dev_key_non_sicura
29
30 docker compose --env-file .env.dev up -d
```

7.5 Docker Bench for Security

Listing 7.5: Audit di sicurezza automatizzato

```
1 # Docker Bench: audit CIS Docker Benchmark
2 docker run -it --net host --pid host --userns host \
3   --cap-add audit_control \
4   -e DOCKER_CONTENT_TRUST=$DOCKER_CONTENT_TRUST \
5   -v /var/lib:/var/lib:ro \
6   -v /var/run/docker.sock:/var/run/docker.sock:ro \
7   -v /usr/lib/systemd:/usr/lib/systemd:ro \
8   -v /etc:/etc:ro \
9   --label docker_bench_security \
10  docker/docker-bench-security
11
12 # Output:
13 # [PASS] 1.1.1 Ensure a separate partition for containers...
14 # [WARN] 2.1 Ensure network traffic is restricted between containers
15 # [FAIL] 4.1 Ensure that a user for the container has been created
16 # [INFO] 5.2 Ensure SELinux security options are set
```

Capitolo 8

Docker e Intelligenza Artificiale

Docker nell'era dell'IA Generativa

Docker è diventato lo standard de facto per il deployment di modelli AI. Nel 2026, il **92% delle aziende IT** usa container per workload AI/ML (Docker Survey 2025; USDSI 2026).

8.1 Perché Docker è ideale per l'AI

Tabella 8.1: Vantaggi di Docker per workload AI/ML

Vantaggio	Dettaglio
Riproducibilità	L'immagine include Python, CUDA, librerie: stessa versione ovunque
Isolamento dipendenze	Ogni modello ha il suo ambiente senza conflitti
GPU passthrough	NVIDIA Container Toolkit passa la GPU al container
Scalabilità	Da laptop a cluster Kubernetes con la stessa immagine
Versionamento	Immagini taggate come i modelli (v1.0, v2.0-quantized)
Sicurezza	Modelli in container isolati dal sistema host

8.2 NVIDIA Container Toolkit e GPU

Il **NVIDIA Container Toolkit** (versione 1.18.0, 2026) permette ai container di accedere alle GPU NVIDIA senza installare i driver CUDA all'interno del container.

Listing 8.1: Installazione NVIDIA Container Toolkit su Debian/Kali

```
1 # Aggiungi repository NVIDIA
2 curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | \
3     sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container.gpg
4
5 DIST=$(. /etc/os-release; echo "${ID}${VERSION_ID}")
6 curl -sL https://nvidia.github.io/libnvidia-container/stable/deb/\
7 nvidia-container-toolkit.list | \
8     sed 's#deb#deb [signed-by=/usr/share/keyrings/\
9 nvidia-container.gpg] #g' | \
10     sudo tee /etc/apt/sources.list.d/nvidia-container.list
11
12 sudo apt update
13sudo apt install -y nvidia-container-toolkit
14
15 # Configura Docker per usare il runtime NVIDIA
16sudo nvidia-ctk runtime configure --runtime=docker
17sudo systemctl restart docker
18
```

```

19 # Verifica: la GPU e' visibile nel container
20 docker run --rm --gpus all nvidia/cuda:12.8-base-ubuntu24.04 \
21     nvidia-smi
22 # Thu Jun 13 10:00:00 2026
23 #
24 # +-----+
25 # | NVIDIA-SMI 550.144      Driver Version: 550.144      CUDA Version: 12.8 |
26 # | GPU Name               Persistence-M| Bus-Id | Volatile Uncorr. ECC |
27 # | 0 NVIDIA RTX 4090 Off | 00000000:01:00.0 Off | Off |
28 # +-----+

```

8.3 Eseguire LLM in locale con Docker

Listing 8.2: Dockerfile per Ollama (LLM locale)

```

1 # Ollama: esegui LLM in locale (Llama3, Mistral, Gemma...)
2 FROM ollama/ollama:latest
3
4 # Pre-scarica un modello leggero durante la build
5 # (opzionale: consuma spazio nell'immagine)
6 # RUN ollama pull llama3.2:3b
7
8 EXPOSE 11434
9
10 # Avvia server Ollama
11 CMD ["ollama", "serve"]

```

Listing 8.3: compose.yaml per stack AI locale con Ollama + Open WebUI

```

1 services:
2
3   # LLM Server (Ollama)
4   ollama:
5     image: ollama/ollama:latest
6     ports:
7       - "11434:11434"
8     volumes:
9       - ollama_models:/root/.ollama # modelli persistenti
10    deploy:
11      resources:
12        reservations:
13          devices:
14            - driver: nvidia
15              count: all # tutte le GPU disponibili
16              capabilities: [gpu]
17    restart: unless-stopped
18    healthcheck:
19      test: ["CMD", "curl", "-f", "http://localhost:11434/api/tags"]
20      interval: 30s
21      timeout: 10s
22      retries: 5

```

```
23
24  # Interfaccia web per Ollama
25  open-webui:
26    image: ghcr.io/open-webui/open-webui:latest
27    ports:
28      - '3000:8080'
29    environment:
30      OLLAMA_BASE_URL: http://ollama:11434
31      WEBUI_SECRET_KEY: cambia_questa_chiave_in_produzione
32    volumes:
33      - webui_data:/app/backend/data
34    depends_on:
35      ollama:
36        condition: service_healthy
37    restart: unless-stopped
38
39  volumes:
40    ollama_models:
41    webui_data:
```

Listing 8.4: Avviare e usare lo stack LLM

```
1  # Avvia lo stack
2  docker compose up -d
3
4  # Scarica un modello (13B parametri, circa 7GB)
5  docker compose exec ollama ollama pull llama3.2:3b
6
7  # Testa l'API direttamente
8  curl http://localhost:11434/api/generate -d '{
9    'model': 'llama3.2:3b',
10   'prompt': 'Spiega Docker in 2 righe',
11   'stream': false
12 }'
13
14 # Apri l'interfaccia web
15 xdg-open http://localhost:3000 # Linux desktop
16 # oppure vai su http://localhost:3000 nel browser
17
18 # Modelli disponibili
19 docker compose exec ollama ollama list
```

8.4 Docker GenAI Stack con NVIDIA NIM

Listing 8.5: compose.yaml con NVIDIA NIM (inference ottimizzata)

```
1  services:
2
3    # NVIDIA NIM: inference ottimizzata per LLM enterprise
4    nim-llama:
5      image: nvcr.io/nim/meta/llama-3.1-8b-instruct:latest
6      ports:
7        - '8000:8000'
8      environment:
9        NGC_API_KEY: ${NGC_API_KEY} # chiave NVIDIA NGC (da .env)
```

```
10     volumes:
11         - nim_cache:/opt/nim/.cache
12     deploy:
13         resources:
14             reservations:
15                 devices:
16                     - driver: nvidia
17                       count: 1
18                       capabilities: [gpu]
19     healthcheck:
20         test: [ 'CMD', 'curl', '-f', 'http://localhost:8000/v1/health/ready' ]
21         interval: 30s
22         timeout: 30s
23         start_period: 120s      # NIM impiega piu' tempo a partire
24         retries: 5
25
26     # RAG pipeline con ChromaDB (vector store)
27     chromadb:
28         image: chromadb/chroma:latest
29         ports:
30             - '8001:8000'
31         volumes:
32             - chroma_data:/chroma/chroma
33         environment:
34             IS_PERSISTENT: 'TRUE'
35
36     # Applicazione RAG (esempio)
37     rag-app:
38         build:
39             context: ./rag_app
40         environment:
41             NIM_BASE_URL: http://nim-llama:8000/v1
42             CHROMA_HOST: chromadb
43             CHROMA_PORT: 8000
44         depends_on:
45             nim-llama:
46                 condition: service_healthy
47             chromadb:
48                 condition: service_started
49         ports:
50             - '5000:5000'
51
52     volumes:
53         nim_cache:
54         chroma_data:
```

8.5 Docker AI Catalog

Nel 2026 Docker ha lanciato il **Docker AI Catalog**: una raccolta curata di immagini AI ottimizzate, pronte all'uso e certificate.

Listing 8.6: Docker AI Catalog: accesso e utilizzo

```
1 # Cerca immagini AI nel catalog
2 docker search --filter is-official=true \
```

```
3  --format 'table {{.Name}}\t{{.Description}}' ai
4
5  # Immagini disponibili (2026):
6  # - qwen3:7b-instruct      (Alibaba, reasoning)
7  # - granite-4.0-nano      (IBM, lightweight)
8  # - gemma3:4b-instruct    (Google)
9  # - phi-4-mini             (Microsoft)
10 # - deepseek-r2-distill   (DeepSeek)
11
12 # Avvia modello dal catalog
13 docker run --gpus all -p 11434:11434 \
14   docker.io/ai/qwen3:7b-instruct
15
16 # API compatibile OpenAI (standard de facto 2026)
17 curl http://localhost:11434/v1/chat/completions \
18   -H 'Content-Type: application/json' \
19   -d '{
20     'model': 'qwen3:7b-instruct',
21     'messages': [
22       {'role': 'user',
23        'content': 'Come funziona Docker internamente?'}
24     ]
25   }'
```

8.6 Containerizzare un modello custom

Listing 8.7: Dockerfile per servire un modello HuggingFace

```
1 FROM python:3.12-slim
2
3 # Installa dipendenze di inferenza
4 RUN pip install --no-cache-dir \
5     transformers==4.44.0 \
6     torch==2.4.0 \
7     accelerate==0.34.0 \
8     fastapi==0.115.0 \
9     uvicorn==0.30.0 \
10    huggingface-hub==0.25.0
11
12 WORKDIR /app
13
14 # Pre-scarica il modello durante la build
15 # (se piccolo; altrimenti usa volume per i pesi)
16 ARG MODEL_NAME=distilbert-base-uncased-finetuned-sst-2-english
17 RUN python -c 'from transformers import pipeline; \
18     pipeline('text-classification', model='${MODEL_NAME})'
19
20 COPY serve.py .
21
22 EXPOSE 8080
23 CMD ['uvicorn', 'serve:app', '--host', '0.0.0.0', '--port', '8080']
```

Listing 8.8: serve.py: API FastAPI per il modello


```
1 # serve.py - server FastAPI per il modello HuggingFace
2 from fastapi import FastAPI
3 from pydantic import BaseModel
4 from transformers import pipeline
5
6 app = FastAPI(title='Sentiment Analysis API')
7
8 # Carica modello una volta sola all'avvio
9 classifier = pipeline(
10     'text-classification',
11     model='distilbert-base-uncased-finetuned-sst-2-english',
12     device=-1 # -1=CPU, 0=prima GPU
13 )
14
15 class Request(BaseModel):
16     text: str
17
18 @app.get('/health')
19 def health(): return {'status': 'ok'}
20
21 @app.post('/predict')
22 def predict(req: Request):
23     result = classifier(req.text)[0]
24     return {
25         'text': req.text,
26         'label': result['label'],
27         'score': round(result['score'], 4)
28     }
```

Listing 8.9: Build e test del modello containerizzato

```
1 # Build
2 docker build -t sentiment-api:latest .
3
4 # Avvio (CPU)
5 docker run -d -p 8080:8080 --name sentiment sentiment-api:latest
6
7 # Avvio con GPU NVIDIA
8 docker run -d -p 8080:8080 --gpus all \
9     --name sentiment sentiment-api:latest
10
11 # Test
12 curl -X POST http://localhost:8080/predict \
13     -H 'Content-Type: application/json' \
14     -d '{"text": "Docker is absolutely amazing!"}'
15
16 # Output:
17 # {'text': 'Docker is absolutely amazing!',
18 #   'label': 'POSITIVE', 'score': 0.9998}
```

Capitolo 9

Sviluppi Futuri ed Ecosistema 2026

9.1 Lo stato dell’ecosistema nel 2026

9.1.1 containerd 2.0: il nuovo runtime

Rilasciato a fine 2024, **containerd 2.0** rappresenta una riscrittura significativa focalizzata su sicurezza e performance.

Listing 9.1: containerd 2.0: nuove funzionalità

```
1 # containerd e' il runtime sottostante a Docker su Linux
2 # Si puo' usare direttamente con nerdctl (alternativa alla CLI Docker)
3 # sudo pacman -S nerdctl oppure sudo apt install nerdctl
4
5 # nerdctl: drop-in replacement di docker CLI
6 nerdctl run --rm hello-world
7 nerdctl compose up -d
8
9 # Sandbox hardware con gVisor (runtime OCI sicuro)
10 # Ogni container ha il suo kernel virtuale (no kernel sharing!)
11 sudo apt install runsc
12 cat /etc/containerd/config.toml # aggiungi runtime gVisor
13 # [plugins.'io.containerd.grpc.v1.cri'.containerd.runtimes.gvisor]
14 #   runtime_type = 'io.containerd.runsc.v1'
15
16 docker run --runtime=runsc --rm nginx # con gVisor
17
18 # Kata Containers: VM leggere come container
19 # Ogni container ha il suo micro-kernel (QEMU/cloud-hypervisor)
20 docker run --runtime=kata-runtime --rm nginx
```

9.1.2 Podman 5.x: alternativa rootless

Listing 9.2: Podman: principali differenze con Docker

```
1 # Podman e' daemonless: non serve un daemon root in background
2 # Ogni container e' un processo figlio dell'utente
3
4 # Installa Podman
5 sudo apt install podman # Debian/Kali
6 sudo pacman -S podman # Arch
7
8 # API Docker-compatibile: stessi comandi!
9 podman run --rm hello-world
10 podman build -t myapp .
11 podman compose up -d # usa compose compatibile
12
13 # Pod: gruppo di container che condividono namespace
```

```
14 podman pod create --name mypod -p 8080:80
15 podman run -d --pod mypod nginx
16 podman run -d --pod mypod redis
17
18 # Genera systemd unit da un container
19 podman generate systemd --name mycontainer > \
20 ~/.config/systemd/user/mycontainer.service
21 systemctl --user enable --now mycontainer
22
23 # Genera kubernetes manifest
24 podman generate kube mypod > mypod.yaml
25 kubectl apply -f mypod.yaml
```

9.2 WebAssembly: la nuova frontiera

Listing 9.3: WASM come alternativa ai container per workload edge

```
1 # WasmEdge: runtime WASM per microservizi (molto piu' leggero)
2 # Avvio: ~1ms vs ~100ms per container
3 # Dimensione: KB vs MB
4
5 # Docker supporta WASM runtime (sperimentale, 2025+)
6 # Richiede: docker desktop con WASM feature flag
7 docker run --runtime=io.containerd.wasmedge.v1 \
8   --platform wasi/wasm \
9   ghcr.io/containerd/runwasi-demo:wasm32-wasi \
10  /hello.wasm
11
12 # Casi d'uso WASM nel 2026:
13 # - Edge computing (IoT, CDN edge functions)
14 # - Plugin sicuri in applicazioni (DB, server)
15 # - Microservizi ultra-leggeri
16 # - Funzioni serverless
17
18 # Confronto:
19 # Container Docker: ~30MB RAM, ~100ms startup
20 # WASM container: ~1MB RAM, ~1ms startup
```

9.3 Docker e Kubernetes

Listing 9.4: Da Docker Compose a Kubernetes

```
1 # Converti compose.yaml in manifest Kubernetes
2 # Tool: kompose (kubernetes.io)
3 sudo apt install kompose # oppure download da github
4
5 cd my_project/
6 kompose convert -f compose.yaml
7
8 # Genera: web-deployment.yaml, db-deployment.yaml, ecc.
9 kubectl apply -f .
10
11 # Oppure installa direttamente con Helm
12 helm repo add bitnami https://charts.bitnami.com/bitnami
```

```
13 helm install myapp bitnami/apache
14
15 # Docker Desktop include Kubernetes (sviluppo locale)
16 kubectl config use-context docker-desktop
17
18 # k3s: Kubernetes leggero per edge/IoT/CI
19 curl -sfl https://get.k3s.io | sh -
```

9.4 Tendenze e sviluppi futuri

Tabella 9.1: Tendenze nell’ecosistema container 2026–2028

Tendenza	Descrizione e impatto
AI-native containers	Immagini ottimizzate per LLM con CUDA/ROCm preinstallato; Docker AI Catalog in espansione
Confidential computing	Container in Trusted Execution Environments (Intel TDX, AMD SEV): dati cifrati anche durante l’uso
eBPF networking	Cilium e Calico eBPF sostituiscono kube-proxy: latenza ridotta del 50%, visibilità totale
Supply chain security	Sigstore/cosign per firma immagini; SBOM (Software Bill of Materials) obbligatorio
WASM maturo	WebAssembly come runtime standard per edge e microservizi ultra-leggeri
Compose v5 SDK	Go SDK permette di integrare Compose nelle applicazioni programmaticamente
Platform engineering	Internal Developer Platform (IDP) con Docker come livello di astrazione standardizzato
Green computing	Ottimizzazione consumo energetico container; metriche CO2 per workload AI

Firma delle immagini con Cosign (Sigstore)

```
# Installa cosign
sudo apt install cosign # oppure download da github.com/sigstore/cosign

# Firma un'immagine dopo il push
docker push myregistry/myapp:latest
cosign sign --key cosign.key myregistry/myapp:latest

# Verifica firma
cosign verify --key cosign.pub myregistry/myapp:latest

# SBOM: genera lista dipendenze
syft myregistry/myapp:latest -o spdx-json > sbom.json
```

9.5 Sintesi: scegliere il tool giusto nel 2026

Tabella 9.2: Guida alla scelta del tool container

Scenario	Tool raccomandato
Sviluppo locale	Docker Desktop o Podman Desktop
Build immagini	Docker BuildKit o Buildah
Multi-container locale	Docker Compose v2/v5
Produzione single-host	Docker Compose + Traefik o Nginx proxy
Produzione multi-host	Kubernetes (k3s, EKS, GKE, AKS)
Sicurezza massima	Podman rootless + gVisor + AppArmor
AI/ML workload	Docker + NVIDIA Container Toolkit + NIM
Edge/IoT	k3s + container leggeri + WASM
CI/CD pipeline	BuildKit con cache remota + registry OCI

Capitolo 10

YAML: Il Linguaggio di Docker Compose

10.1 Che cos'è YAML

YAML (acronimo ricorsivo di *YAML Ain't Markup Language*, inizialmente *Yet Another Markup Language*) è un formato di serializzazione dei dati progettato per essere leggibile dagli esseri umani. È stato creato da Clark Evans nel 2001 e la versione 1.2 del 2009 è quella attualmente in uso.

YAML non è un linguaggio di programmazione, non è un linguaggio di markup come HTML e non gestisce la logica. È semplicemente un modo per **descrivere strutture dati** (dizionari, liste, scalari) in forma testuale. È il formato scelto da Docker Compose, Kubernetes, Ansible, GitHub Actions e molti altri strumenti DevOps proprio per questa sua caratteristica: è immediato da scrivere e capire.

Tabella 10.1: YAML vs JSON vs TOML vs INI a confronto

Formato	Estensione	Leggibilità	Uso principale
YAML	.yaml, .yml	Alta	Docker Compose, Kubernetes, CI/CD
JSON	.json	Media	API REST, configurazioni programmatiche
TOML	.toml	Alta	Rust (Cargo.toml), Python (pyproject.toml)
INI	.ini, .cfg	Alta	Python, Windows, configurazioni semplici
XML	.xml	Bassa	Java, SOAP, formati legacy

10.2 Sintassi YAML: regole fondamentali

10.2.1 Indentazione: la regola più importante

In YAML, la struttura gerarchica è determinata esclusivamente dall'**indentazione con spazi** (mai con tabulazioni). Non esistono parentesi, virgole o tag di chiusura. Questo lo rende estremamente conciso ma anche sensibile agli errori di indentazione.

Listing 10.1: Regole di indentazione YAML

```
1 # CORRETTO: indentazione con 2 spazi (convenzione piu' comune)
2 persona:
3   nome: Mario
4   eta: 30
5   indirizzo:
6     via: Roma 1
```

```
7      citta: Milano
8
9  # CORRETTO: anche 4 spazi sono validi (ma devono essere coerenti)
10 persona:
11     nome: Mario
12     eta: 30
13
14 # ERRORE: tabulazioni non sono permesse in YAML!
15 persona:
16     nome: Mario    # <- TAB: il parser YAML dara' errore
17
18 # ERRORE: indentazione inconsistente
19 persona:
20     nome: Mario
21     eta: 30        # <- 3 spazi invece di 2: errore!
```

10.2.2 Tipi di dati scalari

Listing 10.2: Tipi di dati scalari in YAML

```
1  # -- STRINGHE --
2  stringa_semplice: Hello World
3  stringa_con_virgolette: 'Hello World'      # equivalente
4  stringa_singola: 'Hello World'             # equivalente
5
6  # Stringa con caratteri speciali: usare virgolette
7  url: 'https://example.com/path?a=1&b=2'    # & richiede virgolette
8  path: '/var/log/app.log'                   # / ok senza virgolette
9
10 # Stringa multiriga (pipe |): mantiene i newline
11 messaggio: |
12     Questa e' la prima riga.
13     Questa e' la seconda riga.
14     L'indentazione relativa e' preservata.
15
16 # Stringa multiriga (>) : piega i newline in spazi
17 descrizione: >
18     Questa e' una lunga descrizione
19     che viene unita in una sola riga
20     separata da spazi.
21 # Risultato: 'Questa e' una lunga descrizione che viene unita...'
22
23 # -- NUMERI --
24 intero: 42
25 negativo: -10
26 float: 3.14
27 scientifico: 1.5e10      # 15000000000.0
28 ottale: 0o17             # = 15 in decimale
29 esadecimale: 0xFF        # = 255 in decimale
30
31 # -- BOOLEANI --
32 # ATTENZIONE: solo questi valori sono booleani in YAML 1.2
33 vero: true
34 falso: false
35 # In YAML 1.1 (vecchio) anche: yes/no, on/off, True/False
36 # Docker Compose usa YAML 1.2 -> attenersi a true/false
```

```
37
38 # -- NULL --
39 valore_nullo: null           # equivalente a: ~
40 campo_vuoto:                # anche questo e' null
41
42 # -- DATE (interpretate automaticamente) --
43 data: 2026-06-13           # datetime ISO 8601
44 timestamp: 2026-06-13T10:30:00Z
```

Attenzione

Trappola comune con i booleani: In YAML 1.1 (usato da molti parser legacy) `yes`, `no`, `on`, `off` sono interpretati come booleani. In YAML 1.2 (standard attuale) sono stringhe normali. Docker Compose usa YAML 1.2 ma per sicurezza usare sempre `true/false` tra virgolette dove c'è ambiguità.

10.2.3 Strutture composite: mappe e sequenze

Listing 10.3: Mappe (dizionari) e sequenze (liste) in YAML

```
1 # -- MAPPA (dizionario / oggetto) --
2 # Forma estesa (block style) - preferita per leggibilità
3 database:
4   host: localhost
5   port: 5432
6   name: myapp
7   user: admin
8
9 # Forma inline (flow style) - equivalente, usata per brevità
10 database: {host: localhost, port: 5432, name: myapp, user: admin}
11
12 # -- SEQUENZA (lista / array) --
13 # Forma estesa (ogni elemento preceduto da -)
14 frutti:
15   - mela
16   - pera
17   - banana
18
19 # Forma inline
20 frutti: [mela, pera, banana]
21
22 # -- ANNIDAMENTO: mappa di mappe --
23 applicazione:
24   web:
25     image: nginx:1.25-alpine
26     porta: 8080
27   database:
28     image: postgres:16
29     porta: 5432
30
31 # -- ANNIDAMENTO: lista di mappe --
32 servizi:
33   - nome: web
34     porta: 80
35     attivo: true
36   - nome: api
```



```
37     porta: 8080
38     attivo: false
39
40 # -- LISTA DI LISTE --
41 matrice:
42   - [1, 2, 3]
43   - [4, 5, 6]
44   - [7, 8, 9]
```

10.2.4 Funzionalità avanzate YAML

Listing 10.4: Anchor, alias e merge: riuso dei dati in YAML

```
1  # -- ANCHOR (⌘) e ALIAS (*): riuso di valori --
2  # Definisce un anchor
3  valori_default: &default
4     image: debian:12-slim
5     restart: unless-stopped
6     user: '1000:1000'
7
8  # Usa l'alias (copia tutti i valori dell'anchor)
9  web:
10     <<: *default           # merge dell'anchor
11     image: nginx:alpine    # sovrascrive 'image' dell'anchor
12     ports:
13       - '80:80'
14
15  api:
16     <<: *default           # riusa gli stessi default
17     ports:
18       - '8080:8080'
19
20 # Risultato equivalente:
21 # web:
22 #   image: nginx:alpine    <- sovrascritta
23 #   restart: unless-stopped
24 #   user: '1000:1000'
25 #   ports: ['80:80']
26
27 # -- ANCORA per stringhe --
28 versioni:
29   python: &py_ver '3.12'
30   numpy: '1.26.0'
31
32 ambiente:
33   PYTHON_VERSION: *py_ver  # riusa il valore '3.12'
34
35 # -- DOCUMENTI MULTIPLI nello stesso file --
36 # Separati da ---
37 ---
38 # Documento 1
39 nome: Alice
40 eta: 30
41 ---
42 # Documento 2
43 nome: Bob
```

```
44 eta: 25
45 ...    # fine documento (opzionale)
46
47 # -- COMMENTI --
48 # I commenti iniziano con # e vanno fino a fine riga
49 # Non esistono commenti multiriga in YAML
50 chiave: valore    # commento inline
```

10.3 Perché YAML in Docker Compose

La scelta di YAML per `compose.yaml` non è casuale. Docker Compose gestisce strutture dati gerarchiche complesse: servizi con sotto-configurazioni, reti, volumi, dipendenze. YAML si presta perfettamente a questo tipo di dati per diversi motivi:

Tabella 10.2: Motivazioni della scelta YAML per Docker Compose

Motivo	Spiegazione
Leggibilità umana	La struttura a rientri rispecchia visivamente la gerarchia dei componenti
Gerarchia naturale	Servizi → configurazioni → sotto-opzioni mappano naturalmente in YAML
Multiriga	Il formato <code> </code> e <code>></code> permette script e comandi su più righe senza escape
Tipizzazione	Distingue automaticamente numeri, booleani e stringhe (porta 80 vs stringa "80")
Commenti	A differenza di JSON, YAML supporta commenti (<code>#</code>) per documentare le scelte
Anchor/Alias	Riuso di configurazioni comuni tra servizi senza ripetizioni
Ecosistema	Standard de facto in DevOps: Kubernetes, Ansible, GitHub Actions usano YAML

10.4 Struttura di `compose.yaml` spiegata dall'interno

Listing 10.5: `compose.yaml` analizzato struttura per struttura

```
1 # ==
2 #  STRUTTURA TOP-LEVEL di compose.yaml
3 #  Docker Compose riconosce esattamente queste 5 chiavi top-level:
4 #  services, networks, volumes, configs, secrets
5 # ==
6
7 # -- 1. SERVICES (obbligatorio) --
8 # Mappa YAML: chiave = nome servizio, valore = configurazione
9 services:
10
11   # Il nome "web" è la chiave YAML che diventa:
12   # - il nome DNS interno (altri container lo raggiungono con "web")
13   # - il prefisso del nome container (es: progetto_web_1)
14   web:
15
16     # image: stringa YAML semplice
17     # formato: [registry/][repository/]nome[:tag][@digest]
```

```
18 image: nginx:1.25-alpine      # stringa
19
20 # ports: sequenza YAML di stringhe
21 # formato stringa: 'HOST:CONTAINER[/protocollo]:'
22 ports:
23   - '8080:80'                  # porta host 8080 -> container 80
24   - '443:443'                  # HTTPS
25   - '127.0.0.1:9090:9090'      # solo su loopback
26
27 # environment: puo' essere MAPPA o SEQUENZA
28 # Forma mappa (chiave: valore):
29 environment:
30   NODE_ENV: production
31   PORT: '3000'                  # stringa per evitare ambiguita'
32   DEBUG: 'false'                # stringa, non booleano YAML
33
34 # Forma sequenza (alternativa equivalente):
35 # environment:
36 #   - NODE_ENV=production
37 #   - PORT=3000
38
39 # volumes: sequenza di stringhe o mappe
40 volumes:
41   # Forma stringa breve: HOST:CONTAINER[:opzioni]
42   - ./html:/var/www/html:ro
43   - myvolume:/data
44
45   # Forma mappa estesa (piu' esplicita):
46   - type: bind
47     source: ./config
48     target: /app/config
49     read_only: true
50
51 # depends_on: puo' essere LISTA o MAPPA con condizioni
52 depends_on:
53   # Forma semplice (lista):
54   # - db
55   # - cache
56
57   # Forma avanzata (mappa con condizione):
58   db:
59     condition: service_healthy # attende healthcheck OK
60   cache:
61     condition: service_started # attende solo avvio
62
63 # healthcheck: mappa YAML con sotto-chiavi
64 healthcheck:
65   # test: sequenza YAML (forma exec - preferita)
66   test: ['CMD', 'curl', '-f', 'http://localhost/']
67   # test: ['CMD-SHELL', 'curl -f http://localhost/ || exit 1']
68   interval: 30s                # stringa con unita' di tempo
69   timeout: 5s
70   retries: 3                    # intero YAML
71   start_period: 10s
72
73 # deploy: risorse e replica (Compose e Swarm)
74 deploy:
```

```
75     replicas: 2                # intero: numero istanze
76     resources:
77         limits:
78             cpus: '0.5'        # stringa: 0.5 CPU core
79             memory: 512M      # stringa con unita'
80         reservations:
81             memory: 256M
82
83     # restart: stringa con politica di riavvio
84     # Valori: no / always / on-failure / unless-stopped
85     restart: unless-stopped
86
87     # networks: lista di reti a cui appartiene
88     networks:
89         - frontend
90         - backend
91
92     # logging: mappa con driver e opzioni
93     logging:
94         driver: json-file      # stringa
95         options:               # mappa di stringhe
96             max-size: '10m'
97             max-file: '3'
98
99 # -- 2. NETWORKS (opzionale) --
100 # Mappa: nome_rete -> configurazione (null = default bridge)
101 networks:
102     frontend:                 # rete con configurazione default
103         driver: bridge
104
105     backend:
106         driver: bridge
107         internal: true        # booleano: non raggiungibile da fuori
108
109     # Usa una rete esistente (non la crea)
110     existing_net:
111         external: true
112         name: my_existing_network
113
114 # -- 3. VOLUMES (opzionale) --
115 volumes:
116     myvolume:                 # volume con configurazione default
117         driver: local
118
119     nfs_data:                 # volume NFS
120         driver: local
121         driver_opts:
122             type: nfs
123             o: addr=192.168.1.10,rw
124             device: ':/path/to/nfs'
```

10.5 Errori YAML comuni e come evitarli

Listing 10.6: Errori YAML frequenti in compose.yaml

```

1  # -- ERRORE 1: porta come numero invece di stringa --
2  # SBAGLIATO: il parser interpreta 80:80 come operazione matematica!
3  ports:
4      - 80:80          # ERRORE in alcuni parser YAML
5
6  # CORRETTO: sempre tra virgolette
7  ports:
8      - '80:80'        # OK
9
10 # -- ERRORE 2: booleano non quotato dove si vuole stringa --
11 environment:
12     ENABLE_FEATURE: true      # booleano YAML (true/false)
13     ENABLE_FEATURE: 'true'    # stringa (sicuro per le app)
14     # Alcune app Python leggono 'True' (stringa) diversamente
15     # da True (booleano) - usare sempre virgolette per env vars
16
17 # -- ERRORE 3: anchor merge con campo che sovrascrive --
18 default: &base
19     image: ubuntu
20     restart: always
21
22 web:
23     <<: *base
24     image: nginx            # SOVRASCRIVE image dell'anchor -> OK
25     extra: valore          # AGGIUNGE campo non presente nell'anchor -> OK
26
27 # -- ERRORE 4: tab invece di spazi --
28 # Nei file YAML NON usare mai il tasto TAB
29 # Configurare l'editor per espandere TAB -> spazi:
30 # VSCode: 'editor.insertSpaces': true, 'editor.tabSize': 2
31 # vim: :set expandtab shiftwidth=2 tabstop=2
32
33 # -- ERRORE 5: versione deprecata (comune nei tutorial vecchi) --
34 # NON scrivere piu' (ignorato da Compose v2+):
35 version: '3.8'            # DEPRECATO - campo ignorato
36
37 # CORRETTO: inizia direttamente con services:
38 services:
39     web:
40         image: nginx

```

10.6 Validare un file YAML

Listing 10.7: Strumenti di validazione YAML

```

1  # -- Validazione sintattica con Python --
2  python3 -c 'import yaml; yaml.safe_load(open('compose.yaml'))' \
3      && echo 'YAML valido' || echo 'ERRORE YAML'
4
5  # -- yamllint: linter YAML avanzato --
6  sudo apt install yamllint          # Debian/Kali
7  sudo pacman -S yamllint           # Arch
8
9  yamllint compose.yaml
10 # Output se corretto: (silenzioso)
11 # Output se errore:

```

```
12 # compose.yaml:15:3: [error] wrong indentation (comment)
13 # compose.yaml:23:1: [error] no new line character at the end of file
14
15 # Configurazione yamllint (.yamllint.yml)
16 cat > .yamllint.yml << 'EOF'
17 extends: default
18 rules:
19   line-length:
20     max: 120          # righe max 120 caratteri
21   truthy:
22     allowed-values: ['true', 'false']    # solo lowercase
23   comments:
24     min-spaces-from-content: 1
25 EOF
26 yamllint -c .yamllint.yml compose.yaml
27
28 # -- Docker Compose: validazione nativa --
29 # Controlla la sintassi E la semantica Compose
30 docker compose config          # mostra la config risolta
31 docker compose config --quiet   # solo errori (exit code)
32 docker compose config --services # lista i servizi
33
34 # Risolve variabili d'ambiente e mostra config finale
35 DATABASE_URL=postgres://... docker compose config
36
37 # -- yq: jq per YAML --
38 sudo apt install yq            # Debian/Kali
39 sudo pacman -S go-yq          # Arch
40
41 # Leggi un campo specifico
42 yq '.services.web.image' compose.yaml
43 # Output: nginx:1.25-alpine
44
45 # Lista tutti i nomi dei servizi
46 yq '.services | keys' compose.yaml
47
48 # Modifica inline
49 yq -i '.services.web.image = "nginx:1.26-alpine"' compose.yaml
50
51 # Converti YAML in JSON
52 yq -o=json compose.yaml > compose.json
```

10.7 YAML in altri contesti Docker

Listing 10.8: YAML in Docker Swarm (stack file) e GitHub Actions

```
1 # -- Docker Swarm stack file --
2 # E' identico a compose.yaml con campi extra per Swarm
3 services:
4   web:
5     image: nginx:alpine
6     deploy:                          # solo in Swarm (ignorato da Compose)
7       replicas: 3
8       update_config:
9         parallelism: 2                # aggiorna 2 container alla volta
10        delay: 10s                    # pausa tra aggiornamenti
```

```
11     failure_action: rollback
12     rollback_config:
13         parallelism: 1
14         delay: 5s
15     restart_policy:
16         condition: on-failure
17         delay: 5s
18         max_attempts: 3
19     placement:
20         constraints:
21             - 'node.role == worker' # solo su nodi worker
22             - 'node.labels.zone == eu' # con label specifico
23     configs:
24         - source: nginx_conf # mappa config da secrets Swarm
25           target: /etc/nginx/nginx.conf
26     secrets:
27         - db_password # monta secret come file
28
29     configs:
30         nginx_conf:
31             file: ./nginx.conf # file locale
32
33     secrets:
34         db_password:
35             external: true # creato con: docker secret create
36
37 # -- GitHub Actions (CI/CD con Docker) --
38 # .github/workflows/build.yml
39 name: Build and Push Docker Image
40 on:
41     push:
42         branches: [main]
43
44 jobs:
45     build:
46         runs-on: ubuntu-latest
47         steps:
48             - uses: actions/checkout@v4
49
50             - name: Login to Docker Hub
51               uses: docker/login-action@v3
52               with:
53                 username: ${ secrets.DOCKERHUB_USERNAME }
54                 password: ${ secrets.DOCKERHUB_TOKEN }
55
56             - name: Build and push
57               uses: docker/build-push-action@v5
58               with:
59                 context: .
60                 platforms: linux/amd64,linux/arm64
61                 push: true
62                 tags: myuser/myapp:latest
63                 cache-from: type=gha # cache GitHub Actions
64                 cache-to: type=gha,mode=max
```

Capitolo 11

Cybersecurity e Docker

La sicurezza in un ambiente containerizzato opera su più livelli: la supply chain delle immagini, la configurazione del runtime, la sicurezza della rete e l'hardening del kernel. Questo capitolo copre le tecniche difensive e offensive più rilevanti.

11.1 Threat model: superficie di attacco

Vettori di attacco principali in ambiente Docker	
Vettore	Esempio
Immagine compromessa	Immagine con malware nel registry
Container escape	CVE che rompe l'isolamento namespace
Accesso socket Docker	Escalation via <code>/var/run/docker.sock</code>
Configurazione errata	Container privilegiati o con capabilities eccessive
Segreti nei layer	Password in Dockerfile o variabili ENV
Rete non isolata	Container che si parlano senza restrizioni
Dipendenze vulnerabili	Librerie con CVE nell'immagine

Figura 11.1: Threat model Docker: principali vettori di attacco

11.2 Container escape: tecniche e prevenzione

Il **container escape** è la categoria più critica: un attaccante che ha compromesso un processo nel container riesce ad ottenere accesso all'host.

Listing 11.1: CVE di container escape: esempi storici

```
1 # -- CVE-2019-5736 (runc overwrite) --
2 # Un attaccante con exec in un container poteva sovrascrivere
3 # il binario runc sull'host, ottenendo esecuzione come root.
4 # Fix: runc 1.0.0-rc7, Docker 18.09.2
5
6 # Verifica versione runc
7 runc --version
8 # runc version 1.1.12 (OK, vulnerabilita' corretta)
9
10 # -- CVE-2020-15257 (containerd-shim) --
11 # Accesso a un socket unix esposto in modo non sicuro.
12 # Impattava container con network=host.
13
14 # -- CVE-2022-0492 (cgroup escape) --
15 # Abuso di cgroup v1 release_agent per escape da container
16 # non privilegiati. Fix: kernel 5.17, backport su distro.
17
18 # -- Verifica configurazione sicura --
```



```
19 # Il seguente comando mostra le opzioni di sicurezza attive
20 docker inspect mycontainer --format \
21     '{{json .HostConfig.SecurityOpt}}' | python3 -m json.tool
22
23 # Verifica se un container e' privilegiato (FLAG DI PERICOLO!)
24 docker inspect mycontainer --format \
25     '{{.HostConfig.Privileged}}'
26 # true = PERICOLOSO, false = OK
27
28 # Lista container privilegiati nel sistema
29 docker ps -q | xargs docker inspect \
30     --format '{{.Name}}: privileged={{.HostConfig.Privileged}}' | \
31     grep 'true'
```

11.2.1 Il pericolo del socket Docker

Listing 11.2: Docker socket: vettore di privilege escalation

```
1 # /var/run/docker.sock e' il socket UNIX del daemon Docker.
2 # Chi ha accesso a questo socket HA CONTROLLO COMPLETO DEL HOST!
3
4 # Esempio di escape via socket (NON FARE IN PRODUZIONE):
5 # Un container con accesso al socket puo' creare un container
6 # privilegiato che monta il filesystem host:
7 docker run --rm \
8     -v /var/run/docker.sock:/var/run/docker.sock \
9     docker:cli \
10    docker run --rm -it \
11        --privileged \
12        -v /:/host \
13        ubuntu \
14        chroot /host bash
15 # Risultato: shell root sull'HOST!
16
17 # -- Come proteggere il socket --
18 # 1. Non montare mai docker.sock in produzione (ove possibile)
19 # 2. Se necessario (CI/CD), usare Docker-in-Docker (dind)
20 #     con socket separato e container isolato
21
22 # 3. Usare socket proxy (Tecniche.proxy sicuro)
23 docker run -d \
24     --name socket-proxy \
25     -v /var/run/docker.sock:/var/run/docker.sock \
26     -e CONTAINERS=1 \
27     -e IMAGES=1 \
28     -e NETWORKS=0 \
29     -e VOLUMES=0 \
30     -e POST=0 \           # solo lettura!
31     -p 127.0.0.1:2375:2375 \
32     tecnica/docker-socket-proxy
33
34 # I container usano il proxy invece del socket diretto:
35 # DOCKER_HOST=tcp://socket-proxy:2375
36
37 # 4. Rootless Docker: il socket appartiene all'utente,
38 #     non a root -> compromissione limitata all'utente
```

11.3 Immagini sicure: supply chain security

Listing 11.3: Verifica e firma delle immagini

```
1 # -- Content Trust: verifica firma digitale --
2 # Abilita Docker Content Trust (DCT)
3 export DOCKER_CONTENT_TRUST=1
4
5 # Con DCT attivo, solo immagini firmate possono essere pullate
6 docker pull nginx:latest
7 # Se l'immagine non e' firmata: errore!
8
9 # Firma un'immagine con cosign (Sigstore)
10 # Installa cosign
11 curl -sLO https://github.com/sigstore/cosign/releases/latest/\
12 download/cosign-linux-amd64
13 chmod +x cosign-linux-amd64 && sudo mv cosign-linux-amd64 /usr/bin/cosign
14
15 # Genera chiave di firma
16 cosign generate-key-pair
17
18 # Firma dopo il push
19 docker push myregistry/myapp:v1.0
20 cosign sign --key cosign.key myregistry/myapp:v1.0
21
22 # Verifica firma (in CI/CD o production deployment)
23 cosign verify --key cosign.pub myregistry/myapp:v1.0
24
25 # -- SBOM: Software Bill of Materials --
26 # Genera lista di tutti i pacchetti nell'immagine
27 sudo apt install syft # oppure download da github
28
29 syft myregistry/myapp:v1.0 -o spdx-json > sbom.json
30 syft myregistry/myapp:v1.0 -o table # formato tabella
31
32 # Cerca CVE nell'SBOM con gype
33 sudo apt install gype
34 gype myregistry/myapp:v1.0
35 # Output:
36 # NAME          INSTALLED  FIXED-IN  TYPE    VULNERABILITY  SEVERITY
37 # libssl3       3.0.11     3.0.13    deb     CVE-2024-0727   Medium
38 # openssl       3.0.11     3.0.13    deb     CVE-2024-0727   Medium
39
40 # Analisi completa con Docker Scout
41 docker scout cves myregistry/myapp:v1.0
42 docker scout recommendations myregistry/myapp:v1.0
```

11.3.1 Scegliere immagini base sicure

Tabella 11.1: Immagini base e loro profilo di sicurezza

Immagine base	Dimensione	Profilo sicurezza
scratch	0 MB	Nessuna shell, nessun tool: minima superficie
distroless/*	5-20 MB	Google; nessuna shell, solo runtime
alpine:3.x	8 MB	Musl libc, minimalista, musl CVE rari
debian:12-slim	75 MB	Apt disponibile, piu' compatibile
ubuntu:24.04	100 MB	Completa, molti tool, piu' vulnerabilita'
*:latest	Varia	EVITARE: versione imprevedibile

Listing 11.4: Usare immagini distroless per sicurezza massima

```
1 # Distroless: immagini senza shell, package manager, coreutils
2 # Solo il runtime necessario -> drastica riduzione superficie
3
4 # Stage build normale
5 FROM golang:1.22-alpine AS builder
6 WORKDIR /src
7 COPY . .
8 RUN CGO_ENABLED=0 go build -o /app ./cmd/server
9
10 # Stage runtime: distroless (nessuna shell, nessun bash!)
11 FROM gcr.io/distroless/static-debian12
12 # Non puoi fare: docker exec -it container bash (nessuna shell!)
13 # Non puoi fare: apt install niente
14 # Puoi solo eseguire il binario specificato
15
16 COPY --from=builder /app /app
17 EXPOSE 8080
18 ENTRYPOINT ["/app"]
19
20 # Confronto dimensioni:
21 # Con debian:12-slim: ~120 MB
22 # Con distroless: ~8 MB
23 # Con scratch: ~6 MB (solo il binario)
```

11.4 Runtime security: seccomp, AppArmor, SELinux

Listing 11.5: Profili seccomp personalizzati

```
1 # seccomp (Secure Computing Mode) filtra le syscall
2 # che un container puo' effettuare.
3 # Docker usa un profilo default che blocca ~44 syscall pericolose.
4
5 # Visualizza il profilo seccomp default di Docker
6 docker run --rm alpine cat /proc/1/status | grep Seccomp
7 # Seccomp: 2 (2 = filtro attivo)
8
9 # Verifica il profilo attivo su un container
10 docker inspect mycontainer --format \
```

```

11     '{.HostConfig.SecurityOpt}}'
12
13 # -- Profilo seccomp personalizzato --
14 cat > seccomp-strict.json << 'SECCOMP'
15 {
16     'defaultAction': 'SCMP_ACT_ERRNO',
17     'architectures': ['SCMP_ARCH_X86_64'],
18     'syscalls': [
19         {
20             'names': [
21                 'read', 'write', 'open', 'close', 'stat', 'fstat',
22                 'mmap', 'mprotect', 'munmap', 'brk', 'rt_sigaction',
23                 'rt_sigprocmask', 'ioctl', 'access', 'socket', 'connect',
24                 'sendto', 'recvfrom', 'bind', 'listen', 'accept',
25                 'getsockname', 'getpeername', 'setsockopt', 'getsockopt',
26                 'clone', 'execve', 'wait4', 'kill', 'getpid', 'getuid',
27                 'getgid', 'exit_group', 'futex', 'set_tid_address',
28                 'set_robust_list', 'epoll_create1', 'epoll_ctl',
29                 'epoll_wait', 'openat', 'getdents64'
30             ],
31             'action': 'SCMP_ACT_ALLOW'
32         }
33     ]
34 }
35 SECCOMP
36
37 # Applica il profilo al container
38 docker run \
39     --security-opt seccomp=seccomp-strict.json \
40     --rm myapp:latest
41
42 # Disabilita seccomp (PERICOLOSO - solo per debug)
43 docker run --security-opt seccomp=unconfined --rm myapp:latest
44
45 # -- AppArmor in Docker --
46 # Verifica il profilo AppArmor del container
47 docker inspect mycontainer --format \
48     '{.AppArmorProfile}}'
49 # Output: docker-default (il profilo predefinito)
50
51 # Crea profilo AppArmor custom
52 cat > /etc/apparmor.d/docker-myapp << 'PROFILE'
53 #include <tunables/global>
54
55 profile docker-myapp flags=(attach_disconnected,mediate_deleted) {
56     #include <abstractions/base>
57
58     # Permetti accesso alla rete
59     network tcp,
60     network udp,
61
62     # File system: solo lettura tranne /tmp e /app/data
63     file,
64     deny /etc/shadow r,
65     deny /etc/passwd w,
66     /tmp/** rw,
67     /app/data/** rw,

```

```
68
69     # Capability negate
70     deny capability sys_admin,
71     deny capability sys_ptrace,
72     deny capability net_raw,
73 }
74 PROFILE
75
76 sudo apparmor_parser -r /etc/apparmor.d/docker-myapp
77
78 docker run \
79     --security-opt apparmor=docker-myapp \
80     --rm myapp:latest
```

11.5 Scanning e auditing continuo

Listing 11.6: Pipeline di sicurezza integrata in CI/CD

```
1  # -- Script di audit sicurezza completo --
2  #!/bin/bash
3  # security-audit.sh: audit completo di un'immagine Docker
4
5  IMAGE='${1:-myapp:latest}'
6  FAIL=0
7
8  echo '=== DOCKER SECURITY AUDIT: $IMAGE ==='
9  echo ''
10
11 # 1. Scan vulnerabilita' con Trivy
12 echo '--- [1/5] Vulnerability Scan (Trivy) ---'
13 trivy image --severity HIGH,CRITICAL \
14     --exit-code 1 \
15     '$IMAGE' || FAIL=1
16
17 # 2. Analisi configurazione con Hadolint
18 echo '--- [2/5] Dockerfile Lint (Hadolint) ---'
19 hadolint Dockerfile || FAIL=1
20
21 # 3. Container deve girare come non-root
22 echo '--- [3/5] User check ---'
23 USER=$(docker inspect '$IMAGE' \
24     --format '{{.Config.User}}')
25 if [ -z '$USER' ] || [ '$USER' = 'root' ]; then
26     echo 'FAIL: immagine gira come root!'
27     FAIL=1
28 else
29     echo 'OK: utente = $USER'
30 fi
31
32 # 4. Nessuna password in ENV o LABEL
33 echo '--- [4/5] Secret scan ---'
34 docker inspect '$IMAGE' --format \
35     '{{range .Config.Env}}{{println .}}{{end}}' | \
36     grep -iE 'password|secret|key|token' && {
37     echo 'WARN: possibili segreti in ENV!'
38     FAIL=1
```

```
39 } || echo 'OK: nessun segreto trovato in ENV'
40
41 # 5. Verifica firma (se cosign disponibile)
42 echo '--- [5/5] Image signature ---'
43 if command -v cosign &>/dev/null; then
44     cosign verify --key cosign.pub '$IMAGE' \
45         && echo 'OK: firma valida' \
46         || { echo 'FAIL: immagine non firmata!'; FAIL=1; }
47 else
48     echo 'SKIP: cosign non installato'
49 fi
50
51 echo '...'
52 [ $FAIL -eq 0 ] && echo 'AUDIT PASSATO' || echo 'AUDIT FALLITO'
53 exit $FAIL
```

11.6 Network security: micro-segmentazione

Listing 11.7: Micro-segmentazione di rete in compose.yaml

```
1 # Architettura a zone: ogni zona ha la propria rete
2 # I container comunicano solo con chi devono
3 services:
4
5     # -- DMZ: servizi pubblici --
6     proxy:
7         image: nginx:alpine
8         networks:
9             - dmz          # internet-facing
10            - app_internal # si connette al backend
11         ports:
12             - '443:443'    # unica porta esposta all'esterno!
13
14     # -- APP TIER: applicazione --
15     api:
16         image: myapp:latest
17         networks:
18             - app_internal # parla con proxy
19             - db_internal  # parla con database
20         # NESSUN port mapping: non raggiungibile dall'esterno
21
22     worker:
23         image: myworker:latest
24         networks:
25             - db_internal  # solo database e cache
26             - cache_net
27         # NESSUNA rete verso DMZ o internet
28
29     # -- DATA TIER: persistenza --
30     db:
31         image: postgres:16-alpine
32         networks:
33             - db_internal  # solo dall'app tier
34         # NESSUN port mapping
35
36     cache:
```

```
37     image: redis:7-alpine
38     networks:
39         - cache_net      # isolato su rete dedicata
40
41 networks:
42     dmz:
43         driver: bridge
44     app_internal:
45         driver: bridge
46         internal: false   # ha accesso internet per update/API esterne
47     db_internal:
48         driver: bridge
49         internal: true    # ISOLATA: nessun accesso internet
50     cache_net:
51         driver: bridge
52         internal: true    # ISOLATA
```

11.7 Secrets management avanzato

Listing 11.8: HashiCorp Vault con Docker

```
1  # Vault e' il gold standard per la gestione dei segreti.
2  # I container richiedono i segreti a Vault al runtime,
3  # non li ricevono come env var.
4
5  # Avvia Vault in Docker (dev mode per test)
6  docker run -d \
7      --name vault \
8      --cap-add=IPC_LOCK \
9      -e 'VAULT_DEV_ROOT_TOKEN_ID=dev-root-token' \
10     -e 'VAULT_DEV_LISTEN_ADDRESS=0.0.0.0:8200' \
11     -p 8200:8200 \
12     hashicorp/vault:latest
13
14  # Configura Vault
15  export VAULT_ADDR='http://localhost:8200'
16  export VAULT_TOKEN='dev-root-token'
17
18  # Scrivi un segreto
19  vault kv put secret/myapp \
20      db_password='supersecret' \
21      api_key='abc123xyz'
22
23  # Leggi dal container tramite Vault Agent
24  # vault-agent-config.hcl:
25  cat > vault-agent.hcl << 'HCL'
26  auto_auth {
27      method 'approle' {
28          config = {
29              role_id_file_path  = '/vault/role_id'
30              secret_id_file_path = '/vault/secret_id'
31          }
32      }
33  }
34
35  template {
```

```

36     source      = '/templates/app.env.tpl'
37     destination = '/secrets/app.env'
38 }
39 HCL
40
41 # Template del file .env generato da Vault Agent
42 cat > app.env.tpl << 'TPL'
43 {{ with secret 'secret/myapp' -}}
44 DB_PASSWORD={{ .Data.data.db_password }}
45 API_KEY={{ .Data.data.api_key }}
46 {{- end }}
47 TPL

```

11.8 Incident response: cosa fare dopo una compromissione

Listing 11.9: Procedure di incident response per container

```

1  # -- FASE 1: Contenimento --
2  # Isola immediatamente il container compromesso dalla rete
3  docker network disconnect mynet compromised_container
4
5  # Pausa il container (mantiene lo stato per forensics)
6  docker pause compromised_container
7  # NON usare docker stop: potrebbe cancellare prove in memoria
8
9  # -- FASE 2: Evidenza --
10 # Cattura lo stato del container (filesystem diff)
11 docker diff compromised_container > /forensics/diff.txt
12 # A = aggiunto, C = modificato, D = eliminato
13
14 # Esporta il filesystem del container per analisi
15 docker export compromised_container | \
16     gzip > /forensics/container_fs_$(date +%Y%m%d_%H%M).tar.gz
17
18 # Cattura i log prima che vengano ruotati
19 docker logs compromised_container > /forensics/container.log 2>&1
20
21 # Cattura processi in esecuzione nel container
22 docker top compromised_container aux > /forensics/processes.txt
23
24 # Cattura connessioni di rete
25 nsenter -t $(docker inspect compromised_container \
26     --format '{{.State.Pid}}') -n \
27     ss -tulnp > /forensics/network.txt
28
29 # Hash del filesystem per chain of custody
30 sha256sum /forensics/* > /forensics/checksums.sha256
31
32 # -- FASE 3: Analisi --
33 # Analizza il layer scrivibile del container
34 docker inspect compromised_container \
35     --format '{{.GraphDriver.Data.UpperDir}}'
36 # Esamina i file modificati:
37 ls -la /var/lib/docker/overlay2/<ID>/diff/
38
39 # -- FASE 4: Eradicazione e ripristino --

```



```
40 # Rimuovi il container compromesso
41 docker rm -f compromised_container
42
43 # Rimuovi l'immagine se compromessa
44 docker rmi compromised_image:tag
45
46 # Controlla altri container che usano la stessa immagine
47 docker ps -a --filter ancestor=compromised_image:tag
48
49 # Rideploy da immagine verificata
50 docker compose down && docker compose pull && docker compose up -d
```

Capitolo 12

Comandi Avanzati e Opzioni Raramente Usate

Questo capitolo raccoglie comandi, flag e tecniche che raramente compaiono nei tutorial di base ma che risultano preziosi in scenari reali di produzione, debug e ottimizzazione.

12.1 docker build: opzioni avanzate

Listing 12.1: Flag avanzati di docker build

```
1 # -- Target specifico in multi-stage build --
2 # Fermati a uno stage intermedio (utile per debug)
3 docker build --target builder -t myapp:debug .
4 # Entra nel container per ispezionare lo stage
5 docker run --rm -it myapp:debug sh
6
7 # -- Argomenti di build --
8 docker build \
9     --build-arg NODE_ENV=staging \
10    --build-arg API_URL=https://staging.api.com \
11    --build-arg BUILD_DATE=$(date -u +%Y-%m-%dT%H:%M:%SZ) \
12    -t myapp:staging .
13
14 # -- Squash: unifica tutti i layer in uno --
15 # Riduce le dimensioni ma elimina la cache granulare
16 # Richiede: daemon con --experimental
17 docker build --squash -t myapp:slim .
18
19 # -- Nessuna cache per step specifici --
20 # Utile quando un RUN scarica dati che cambiano
21 docker build --no-cache --target deps -t myapp:fresh .
22
23 # -- Output diversi (BuildKit) --
24 # Esporta il filesystem direttamente su disco (senza immagine)
25 docker build --output type=local,dest=./build-output .
26
27 # Esporta come archivio tar
28 docker build --output type=tar,dest=./app.tar .
29
30 # Esporta come immagine OCI
31 docker build --output type=oci,dest=./app-oci.tar .
32
33 # -- Attestations e SBOM (BuildKit avanzato) --
34 docker build \
35     --attest type=sbom \
36     --attest type=provenance,mode=max \
37     --push \
```

```
38     -t myregistry/myapp:latest .
39
40 # Ispeziona le attestazioni
41 docker buildx imagetools inspect myregistry/myapp:latest \
42     --format '{{json .SBOM.SPDX}}'
43
44 # -- SSH forwarding durante la build --
45 # Consente accesso a repository privati SSH senza copiare chiavi
46 eval $(ssh-agent)
47 ssh-add ~/.ssh/id_ed25519
48
49 docker build \
50     --ssh default=$SSH_AUTH_SOCK \
51     -t myapp:private .
52
53 # Nel Dockerfile:
54 # RUN --mount=type=ssh \
55 #     git clone git@github.com:private/repo.git
```

12.2 docker run: flag raramente usati

Listing 12.2: Opzioni avanzate di docker run

```
1 # -- Limite PID (prevenzione fork bomb) --
2 docker run --pids-limit 100 myapp:latest
3 # Massimo 100 processi nel container
4
5 # -- Limite file descriptor --
6 docker run --ulimit nofile=1024:1024 myapp:latest
7 # Limita file aperti a 1024
8
9 # -- Limite dimensione log --
10 docker run \
11     --log-driver json-file \
12     --log-opt max-size=10m \
13     --log-opt max-file=3 \
14     myapp:latest
15
16 # -- SHM size (shared memory) --
17 # Utile per browser headless (Selenium, Puppeteer)
18 docker run --shm-size=2g selenium/standalone-chrome
19
20 # -- Init process (tini) --
21 # Aggiunge un init process leggero (PID 1) che gestisce
22 # correttamente i zombie e i segnali
23 docker run --init myapp:latest
24 # Equivalente a usare tini nel Dockerfile
25
26 # -- Cgroups: swap limit --
27 docker run \
28     --memory=512m \
29     --memory-swap=1g \           # RAM + swap = 1GB totale
30     --memory-swappiness=0 \     # non usare swap se possibile
31     myapp:latest
32
33 # -- CPU pinning: CPU specifiche --
```

```
34 docker run --cpuset-cpus='0,1' myapp:latest
35 # Usa solo CPU 0 e 1
36 docker run --cpuset-cpus='0-3' myapp:latest
37 # Usa CPU da 0 a 3
38
39 # -- CPU shares (peso relativo) --
40 docker run --cpu-shares=512 low-priority-app:latest
41 docker run --cpu-shares=1024 high-priority-app:latest
42 # CPU default = 1024. Se c'è contesa, high-priority riceve il doppio
43
44 # -- IO weight --
45 docker run --blkio-weight=300 myapp:latest
46 # Peso I/O: 10-1000 (default 500)
47
48 docker run \
49     --device-read-bps /dev/sda:10mb \    # max 10 MB/s lettura
50     --device-write-bps /dev/sda:5mb \    # max 5 MB/s scrittura
51     myapp:latest
52
53 # -- Capabilities granulari --
54 # Lista completa delle capability disponibili
55 docker run --rm alpine cat /proc/1/status | grep CapEff
56
57 # Aggiungi solo quello che serve
58 docker run \
59     --cap-drop ALL \
60     --cap-add NET_BIND_SERVICE \    # bind porta < 1024
61     --cap-add SYS_TIME \            # modifica orologio di sistema
62     --cap-add AUDIT_WRITE \         # scrittura audit log
63     myapp:latest
64
65 # -- Runtime alternativo --
66 docker run --runtime=nvidia --gpus all myapp-gpu:latest
67 docker run --runtime=runc myapp:latest    # gVisor
68 docker run --runtime=kata-runtime myapp:latest    # Kata Containers
69
70 # -- Namespace sharing --
71 # Condividi namespace PID con l'host (debug avanzato)
72 docker run --pid=host --rm ubuntu ps aux | head -20
73
74 # Condividi namespace di rete con un altro container
75 docker run --network=container:myapp debug-tools:latest
76 # Ora puoi fare tcpdump sul traffico di myapp!
77
78 # -- Variabili d'ambiente da file --
79 docker run --env-file .env.production myapp:latest
80
81 # -- Hostname e DNS personalizzati --
82 docker run \
83     --hostname myapp.local \
84     --add-host db.internal:192.168.1.10 \
85     --dns 1.1.1.1 \
86     --dns-search internal.company.com \
87     myapp:latest
```

12.3 docker inspect: estrazione informazioni avanzata

Listing 12.3: docker inspect: template Go avanzati

```
1 # docker inspect usa il template engine di Go per l'output.
2 # Sintassi: --format '{{.Campo}}'
3
4 # -- Informazioni base --
5 # IP del container
6 docker inspect myapp --format '{{.NetworkSettings.IPAddress}}'
7
8 # Tutti gli IP (multi-network)
9 docker inspect myapp --format \
10   '{{range $net,$conf := .NetworkSettings.Networks}}{{$net}}: {{$conf.
11     IPAddress}}{{'\n'}}{{end}}'
12
13 # Porta mappata
14 docker inspect myapp --format \
15   '{{range $port,$bind := .NetworkSettings.Ports}}{{$port}} -> {{(index
16     $bind 0).HostPort}}{{'\n'}}{{end}}'
17
18 # -- Status e restart --
19 docker inspect myapp --format \
20   '{{.State.Status}} (restart count: {{.RestartCount}})'
21
22 # Exit code dell'ultimo arresto
23 docker inspect myapp --format \
24   '{{.State.ExitCode}}'
25
26 # Data di avvio e tempo in esecuzione
27 docker inspect myapp --format \
28   'Started: {{.State.StartedAt}}'
29
30 # -- Volumi e bind mount --
31 docker inspect myapp --format \
32   '{{range .Mounts}}{{.Type}}: {{.Source}} -> {{.Destination}} ({{.Mode}})
33     {{'\n'}}{{end}}'
34
35 # -- Variabili d'ambiente --
36 docker inspect myapp --format \
37   '{{range .Config.Env}}{{println .}}{{end}}' | sort
38
39 # -- Confronta due container --
40 # Differenze tra configurazioni
41 diff \
42   <(docker inspect container1 | python3 -m json.tool) \
43   <(docker inspect container2 | python3 -m json.tool)
44
45 # -- Ispezione immagine --
46 # Comando di avvio default
47 docker inspect nginx:latest --format '{{.Config.Cmd}}'
48 docker inspect nginx:latest --format '{{.Config.Entrypoint}}'
49
50 # Porte esposte dall'immagine
51 docker inspect nginx:latest --format \
52   '{{range $p,$ := .Config.ExposedPorts}}{{$p}}{{'\n'}}{{end}}'
```

```
51 # Dimensione totale (somma tutti i layer)
52 docker inspect myapp:latest --format \
53     '{{.Size}} bytes ({{div .Size 1048576}} MB)'
```

12.4 docker events: monitoraggio in tempo reale

Listing 12.4: docker events: stream di eventi del daemon

```
1 # Monitora tutti gli eventi Docker in tempo reale
2 docker events
3
4 # Filtra per tipo di evento
5 docker events --filter type=container
6 docker events --filter type=image
7 docker events --filter type=network
8 docker events --filter type=volume
9
10 # Filtra per azione specifica
11 docker events --filter event=start
12 docker events --filter event=die
13 docker events --filter event=oom # Out of Memory!
14 docker events --filter event=health_status
15
16 # Filtra per container specifico
17 docker events --filter container=myapp
18
19 # Formato output personalizzato
20 docker events --format '{{json .}}' | python3 -m json.tool
21
22 docker events --format \
23     '{{.Time}} [{{.Type}}] {{.Actor.Attributes.name}}: {{.Action}}'
24
25 # Finestra temporale (audit degli ultimi 30 minuti)
26 docker events \
27     --since '30m' \
28     --until 'now' \
29     --filter event=die
30
31 # Integrazione con alerting (esempio con webhook)
32 docker events --filter event=oom --format '{{json .}}' | \
33     while read event; do
34         curl -X POST https://hooks.slack.com/... \
35             -H 'Content-Type: application/json' \
36             -d '{"text": "OOM event: $event"}'
37     done
```

12.5 docker cp, diff e commit

Listing 12.5: Comandi di ispezione e modifica container

```
1 # -- docker cp: copia file in entrambe le direzioni --
2 # Dal container all'host
3 docker cp myapp:/etc/nginx/nginx.conf ./nginx-backup.conf
4 docker cp myapp:/var/log/app/ ./logs-backup/
```

```
5
6 # Dall'host al container (anche in esecuzione!)
7 docker cp ./new-config.conf myapp:/etc/myapp/config.conf
8 docker cp ./hotfix.py myapp:/app/hotfix.py
9
10 # -- docker diff: cambiamenti al filesystem --
11 # Mostra tutti i file modificati rispetto all'immagine base
12 docker diff myapp
13 # A /app/logs/app.log      <- Aggiunto
14 # C /etc/myapp/config      <- Changed (modificato)
15 # D /tmp/tempfile         <- Deleted (eliminato)
16
17 # Utile per:
18 # - Debugging: cosa ha scritto il container?
19 # - Sicurezza: file inaspettati aggiunti da malware?
20 # - Build: quali file sono necessari?
21
22 # -- docker commit: crea immagine da container --
23 # ATTENZIONE: pratica sconsigliata in produzione!
24 # Usare Dockerfile per riproducibilit .
25 # Utile solo per: debug, prototipazione rapida.
26
27 # Commit di un container modificato
28 docker commit \
29   --author 'OTHack1A' \
30   --message 'Aggiunta configurazione debug' \
31   myapp \
32   myapp:debug-$(date +%Y%m%d)
33
34 # Commit con configurazione modificata
35 docker commit \
36   --change 'CMD ["python', '-m', 'debugpy', '--listen',
37     '0.0.0.0:5678', 'app.py']' \
38   --change 'EXPOSE 5678' \
39   myapp \
40   myapp:debugpy
41
42 # -- docker export/import vs docker save/load --
43 # export/import: esporta il FILESYSTEM del container (singolo layer)
44 docker export myapp > myapp-fs.tar
45 docker import myapp-fs.tar myapp:imported
46 # Risultato: immagine senza storia dei layer, senza metadati
47
48 # save/load: esporta l'IMMAGINE completa (tutti i layer + metadati)
49 docker save myapp:latest | gzip > myapp-image.tar.gz
50 docker load < myapp-image.tar.gz
51 # Risultato: immagine identica all'originale, con storia completa
52
53 # Transfer immagine senza registry
54 docker save myapp:latest | \
55   ssh user@server 'docker load'
56 # Oppure con compressione:
57 docker save myapp:latest | gzip | \
58   ssh user@server 'gunzip | docker load'
```

12.6 Comandi di sistema e manutenzione

Listing 12.6: Manutenzione avanzata del sistema Docker

```
1 # -- docker system df: uso spazio dettagliato --
2 docker system df
3 docker system df -v # dettaglio per ogni oggetto
4
5 # Output:
6 # TYPE          TOTAL  ACTIVE  SIZE      RECLAIMABLE
7 # Images        23     5       4.123GB   3.456GB (83%)
8 # Containers    3      3       12.5kB    0B (0%)
9 # Local Volumes 8       4       2.345GB   1.234GB (52%)
10 # Build Cache   145    0       3.456GB   3.456GB (100%)
11
12 # -- Pulizia selettiva per eta' --
13 # Rimuovi build cache piu' vecchia di 48 ore
14 docker builder prune --filter until=48h
15
16 # Rimuovi immagini non usate da piu' di 7 giorni
17 docker image prune -a --filter until=168h
18
19 # Rimuovi container fermati piu' di 24 ore fa
20 docker container prune --filter until=24h
21
22 # -- docker info: diagnostica del daemon --
23 docker info
24 docker info --format '{{json .}}' | python3 -m json.tool
25
26 # Informazioni specifiche
27 docker info --format '{{.MemTotal}}' # RAM totale
28 docker info --format '{{.NCPU}}' # CPU disponibili
29 docker info --format '{{.DockerRootDir}}' # dove Docker salva i dati
30
31 # -- Compressione layer: manipolazione avanzata --
32 # Analisi layer con skopeo (tool alternativo)
33 sudo apt install skopeo
34 skopeo inspect docker://nginx:latest
35 skopeo copy docker://nginx:latest oci:./nginx-oci:latest
36
37 # -- Gestione plugin Docker --
38 docker plugin ls
39 docker plugin install grafana/loki-docker-driver:latest \
40 --alias loki \
41 --grant-all-permissions
42
43 # Usa il plugin come log driver
44 docker run \
45 --log-driver loki \
46 --log-opt loki-url='http://localhost:3100/loki/api/v1/push' \
47 myapp:latest
48
49 # -- Docker checkpoint (CRIU): pause/resume --
50 # Salva lo stato completo del container su disco e riprendilo dopo
51 # Richiede: kernel con CRIU abilitato (sperimentale)
52 docker checkpoint create myapp checkpoint1
53 docker start --checkpoint checkpoint1 myapp
```


12.7 BuildKit: funzionalità avanzate poco conosciute

Listing 12.7: Mount types avanzati in Dockerfile (BuildKit)

```
1  # -- Cache mount: cache persistente tra build --
2  # La cache di apt/pip/npm NON viene invalidata ad ogni build!
3  RUN --mount=type=cache,target=/var/cache/apt,sharing=locked \
4      --mount=type=cache,target=/var/lib/apt,sharing=locked \
5          apt-get update && apt-get install -y \
6          build-essential \
7          libssl-dev
8
9  # pip con cache
10 RUN --mount=type=cache,target=/root/.cache/pip \
11     pip install -r requirements.txt
12
13 # npm con cache
14 RUN --mount=type=cache,target=/root/.npm \
15     npm ci --prefer-offline
16
17 # cargo (Rust) con cache
18 RUN --mount=type=cache,target=/usr/local/cargo/registry \
19     --mount=type=cache,target=/app/target \
20     cargo build --release
21
22 # -- Tmpfs mount: filesystem temporaneo in RAM --
23 # I file scritti qui non finiscono nel layer!
24 RUN --mount=type=tmpfs,target=/tmp \
25     ./configure-with-temp-files.sh
26 # /tmp non viene incluso nell'immagine finale
27
28 # -- Bind mount dal contesto di build --
29 # Monta file dal contesto senza copiarli nel layer
30 RUN --mount=type=bind,source=./scripts,target=/build/scripts \
31     /build/scripts/install.sh
32 # scripts/ non e' nell'immagine finale!
33
34 # -- Heredoc in Dockerfile (BuildKit moderno) --
35 # Crea file inline senza echo e senza file separati
36 COPY <<EOF /etc/nginx/conf.d/default.conf
37 server {
38     listen 80;
39     location / {
40         proxy_pass http://app:3000;
41     }
42 }
43 EOF
44
45 # Script Python inline
46 RUN python3 <<PYEOF
47 import json, os
48
49 config = {
50     'env': os.getenv('APP_ENV', 'production'),
51     'version': os.getenv('APP_VERSION', 'unknown')
52 }
53
```

```
54 with open('/app/config.json', 'w') as f:
55     json.dump(config, f)
56 PYEOF
```

Capitolo 13

Diagnostica e Troubleshooting Avanzato

13.1 Metodologia di debug

Il debugging di problemi Docker segue una progressione logica: dal semplice al complesso, dall'esterno all'interno.

Listing 13.1: Checklist di diagnostica rapida

```
1 # -- PASSO 1: Stato generale --
2 docker ps -a          # tutti i container (anche fermi)
3 docker ps -a --format '{{.Names}}\t{{.Status}}\t{{.Ports}}'
4
5 # -- PASSO 2: Log del container problematico --
6 docker logs myapp
7 docker logs myapp --tail=100 # ultime 100 righe
8 docker logs myapp --since=10m # ultimi 10 minuti
9 docker logs myapp 2>&1 | grep -iE 'error|exception|fatal'
10
11 # -- PASSO 3: Stato dettagliato --
12 docker inspect myapp | python3 -m json.tool | head -80
13
14 # Exit code del container fermato
15 docker inspect myapp --format '{{.State.ExitCode}}'
16 # 0 = terminato normalmente
17 # 1 = errore generico applicazione
18 # 137 = 128+9 = ucciso da SIGKILL (OOM o docker kill)
19 # 139 = 128+11 = segmentation fault
20 # 143 = 128+15 = ucciso da SIGTERM (docker stop)
21
22 # -- PASSO 4: Risorse --
23 docker stats --no-stream
24 docker stats myapp --format \
25     '{{.Name}}: CPU={{.CPUPerc}} MEM={{.MemUsage}}'
26
27 # -- PASSO 5: Rete --
28 docker network inspect mynet
29 docker exec myapp ping -c3 db          # ping tra container
30 docker exec myapp nslookup db          # DNS interno
31 docker exec myapp curl -v http://api:8080/health # HTTP
```

13.2 Container che non si avvia

Listing 13.2: Debug di container che crashano all'avvio

```

1  # Il container esce subito? Testa il comando manualmente:
2  docker run --rm -it myapp:latest sh
3  # Se sh non esiste (distroless):
4  docker run --rm -it --entrypoint /bin/bash myapp:latest
5  # Oppure sovrascrive completamente:
6  docker run --rm -it --entrypoint '"" myapp:latest /bin/bash
7
8  # Debug con entrypoint minimale
9  docker run --rm -it \
10     --entrypoint '"" \
11     myapp:latest \
12     /bin/sh -c 'ls -la /app && cat /app/config.yaml && /app/server --help'
13
14  # Simula le variabili d'ambiente di produzione
15  docker run --rm -it \
16     --env-file .env.production \
17     --entrypoint '"" \
18     myapp:latest \
19     /bin/sh
20
21  # Verifica che il file di avvio esista e sia eseguibile
22  docker run --rm myapp:latest ls -la /app/server
23  # -rwxr-xr-x 1 root root 8234567 /app/server <- OK
24  # -rw-r--r-- 1 root root 8234567 /app/server <- PROBLEMA: non eseguibile!
25
26  # Verifica le dipendenze del binario
27  docker run --rm myapp:latest ldd /app/server
28  # libssl.so.3 => not found <- dipendenza mancante!
29
30  # -- Problemi comuni e soluzioni --
31  # Problema: 'exec format error'
32  # Causa: immagine compilata per architettura sbagliata
33  docker inspect myapp:latest --format '{{.Architecture}}'
34  # amd64 vs arm64 -> usare buildx multi-platform
35
36  # Problema: 'permission denied' su file di avvio
37  # Soluzione nel Dockerfile:
38  # RUN chmod +x /app/server
39
40  # Problema: 'no such file or directory' per librerie
41  # Soluzione: dipendenze mancanti nell'immagine
42  # RUN ldd /app/server | grep 'not found' | awk '{print $1}' | xargs apt-
    get install

```

13.3 Problemi di rete

Listing 13.3: Debug connettività tra container

```

1  # -- Tool di rete nel container --
2  # Molte immagini slim non hanno ping, curl, netstat...
3  # Usa un container 'toolbox' temporaneo sulla stessa rete:
4
5  docker run --rm -it \
6     --network myproject_default \
7     nicolaka/netshoot \
8     bash

```

```
9 # nicolaka/netshoot e' un'immagine ricca di tool di rete:
10 # ping, dig, curl, netstat, ss, tcpdump, iperf3, nmap...
11
12 # -- Testa DNS interno --
13 # Nel container toolbox:
14 nslookup web # risolve il nome del servizio
15 nslookup web.myproject_default # nome FQDN interno
16 dig web +short
17
18 # -- Verifica porte in ascolto --
19 # Dal container che deve connettersi:
20 nc -zv db 5432 # TCP connect test
21 nc -u -zv db 53 # UDP test (DNS)
22
23 # -- tcpdump: cattura traffico tra container --
24 # Installa sul container (Debian):
25 docker exec -it myapp \
26     bash -c 'apt-get update && apt-get install -y tcpdump && tcpdump -i eth0
27     '
28
29 # Oppure usa nsenter per catturare dall'host:
30 PID=$(docker inspect myapp --format '{{.State.Pid}}')
31 sudo nsenter -t $PID -n \
32     tcpdump -i eth0 -nn host db and port 5432
33
34 # -- Ispeziona le iptables rules di Docker --
35 sudo iptables -L DOCKER -n -v --line-numbers
36 sudo iptables -L DOCKER -USER -n -v
37 sudo iptables -t nat -L DOCKER -n
38
39 # -- Problemi comuni di rete --
40 # Problema: container non si raggiungono per nome
41 # Verifica: sono sulla stessa rete Docker?
42 docker network inspect mynet \
43     --format '{{range .Containers}}{{.Name}}: {{.IPv4Address}}{{'\n'}}{{end
44     }}'
45
46 # Problema: porta non raggiungibile dall'host
47 # Verifica: il port mapping e' corretto?
48 docker port myapp
49 docker inspect myapp \
50     --format '{{range $p,$b := .NetworkSettings.Ports}}{{$p}} -> {{$b}}{{'\n
51     '}}{{end}}'
52
53 # Problema: conflitto di subnet tra Docker e VPN
54 docker network ls
55
56 # Spesso la subnet 172.17.0.0/16 di Docker confligge
57 # Soluzione in /etc/docker/daemon.json:
58 # {'bip': '192.168.200.1/24', 'default-address-pools': [
59 #     {'base': '192.168.201.0/24', 'size': 28}]}
```

13.4 Problemi di performance

Listing 13.4: Profiling e ottimizzazione performance container

```
1 # -- Benchmark CPU container --
2 docker run --rm -it \
3   --cpus 1.0 \
4   ubuntu \
5   bash -c ‘apt-get install -y sysbench -q && \
6             sysbench cpu --cpu-max-prime=20000 run‘
7
8 # -- Benchmark I/O disco --
9 docker run --rm -it ubuntu bash -c \
10   ‘dd if=/dev/zero of=/tmp/test bs=1M count=1024 \
11     conv=fsync 2>&1 | tail -1‘
12 # 1073741824 bytes (1.1 GB) copied, 2.34 s, 459 MB/s
13
14 # -- Monitoraggio dettagliato risorse --
15 # Con cAdvisor (Google): monitoring completo via web UI
16 docker run -d \
17   --name cadvisor \
18   --volume=:/rootfs:ro \
19   --volume=/var/run:/var/run:ro \
20   --volume=/sys:/sys:ro \
21   --volume=/var/lib/docker:/var/lib/docker:ro \
22   --publish=8080:8080 \
23   --detach=true \
24   gcr.io/cadvisor/cadvisor:latest
25 # Apri http://localhost:8080
26
27 # -- Profiling con perf (Linux) --
28 PID=$(docker inspect myapp --format ‘{{.State.Pid}}’)
29 sudo perf top -p $PID
30
31 # -- Strace di un processo nel container --
32 docker run --rm -it \
33   --cap-add SYS_PTRACE \
34   --security-opt seccomp=unconfined \
35   myapp:latest \
36   strace -p 1 -e trace=file 2>&1 | head -50
37
38 # -- Latenza I/O overlay2 --
39 # overlay2 puo’ avere overhead sulle scritture intensive
40 # Soluzione: usa volumi per dati scritti frequentemente
41
42 # LENTO: scrittura nel layer container (overlay2)
43 docker run --rm ubuntu bash -c ‘time dd if=/dev/zero of=/data bs=1M count
44   =100‘
45
46 # VELOCE: scrittura in volume (filesystem nativo)
47 docker run --rm -v /tmp/bench:/data ubuntu \
48   bash -c ‘time dd if=/dev/zero of=/data/test bs=1M count=100‘
49
50 # -- Memory profiling --
51 # Uso memoria dettagliato (RSS, cache, swap)
52 docker exec myapp cat /sys/fs/cgroup/memory.current
53 docker exec myapp cat /sys/fs/cgroup/memory.stat
54 # cache = page cache (liberabile)
55 # rss = memoria del processo (non liberabile)
56 # mapped_file = file mappati in memoria
```

13.5 Debug avanzato con nsenter

Listing 13.5: nsenter: entrare nei namespace del container

```
1 # nsenter permette di entrare in uno o piu' namespace
2 # del container senza bisogno di bash nell'immagine!
3
4 # Ottieni il PID del processo principale del container
5 PID=$(docker inspect myapp --format '{{.State.Pid}}')
6 echo "PID: $PID"
7
8 # -- Entra nel namespace di rete --
9 sudo nsenter -t $PID --net \
10     ss -tulnp
11 # Vedi le porte in ascolto nel container senza exec!
12
13 # -- Entra nel namespace PID e mount --
14 sudo nsenter -t $PID --pid --mount \
15     ls -la /proc/1/exe
16 # Mostra il binario principale del container
17
18 # -- Shell completa nei namespace del container --
19 sudo nsenter -t $PID -m -u -i -n -p \
20     /bin/bash
21 # Equivale a essere dentro il container ma come processo esterno
22 # (sei root indipendentemente dall'USER del Dockerfile)
23
24 # -- Leggi file nel filesystem del container --
25 sudo nsenter -t $PID --mount \
26     cat /app/config.yaml
27
28 # -- Debug container distroless (nessuna shell!) --
29 # Con nsenter puoi usare gli strumenti dell'HOST nel container:
30 sudo nsenter -t $PID -m -n \
31     strace -p $PID -e trace=network 2>&1 | head -20
32
33 # -- Ispeziona /proc del processo container --
34 # Mappa di memoria del processo
35 cat /proc/$PID/maps | head -20
36
37 # File descriptor aperti
38 ls -la /proc/$PID/fd
39
40 # Connessioni di rete (hex -> dec per IP e porte)
41 cat /proc/$PID/net/tcp
```

13.6 Log aggregation e analisi

Listing 13.6: Analisi avanzata dei log Docker

```
1 # -- Formato JSON dei log --
2 # Con il driver json-file, i log sono in JSON su disco:
3 cat $(docker inspect myapp \
4     --format '{{.LogPath}}') | head -5
5 # {'log': 'Server started on :8080\n', 'stream': 'stdout',
```

```
6 # 'time': '2026-06-13T10:00:00.123456789Z'
7
8 # Analisi con jq
9 cat $(docker inspect myapp --format '{{.LogPath}}') | \
10 jq -r '.log' | grep ERROR
11
12 # Log solo nell'ultima ora
13 cat $(docker inspect myapp --format '{{.LogPath}}') | \
14 jq -r 'select(.time > '2026-06-13T09:00:00Z') | .log'
15
16 # -- Loki + Grafana: log centralizzati --
17 # Stack minimo per log aggregation
18
19 cat > compose-logging.yaml << 'YAML'
20 services:
21   loki:
22     image: grafana/loki:latest
23     ports: ['3100:3100']
24     volumes:
25       - loki_data:/loki
26     command: -config.file=/etc/loki/local-config.yaml
27
28   grafana:
29     image: grafana/grafana:latest
30     ports: ['3000:3000']
31     environment:
32       GF_AUTH_ANONYMOUS_ENABLED: 'true'
33     volumes:
34       - grafana_data:/var/lib/grafana
35
36 # Promtail: raccoglie i log Docker e li invia a Loki
37 promtail:
38   image: grafana/promtail:latest
39   volumes:
40     - /var/lib/docker/containers:/var/lib/docker/containers:ro
41     - /var/run/docker.sock:/var/run/docker.sock
42   command: -config.file=/etc/promtail/config.yml
43
44 volumes:
45   loki_data:
46   grafana_data:
47 YAML
48
49 docker compose -f compose-logging.yaml up -d
50
51 # -- Configurare i container per inviare log a Loki --
52 # In /etc/docker/daemon.json:
53 # {
54 #   'log-driver': 'loki',
55 #   'log-opts': {
56 #     'loki-url': 'http://localhost:3100/loki/api/v1/push',
57 #     'loki-batch-size': '400'
58 #   }
59 # }
```


13.7 Fonti per il troubleshooting

Docker Engine troubleshooting <https://docs.docker.com/engine/daemon/troubleshoot/>

Guida ufficiale Docker per problemi comuni del daemon.

nicolaka/netshoot <https://github.com/nicolaka/netshoot>

Container toolbox con oltre 50 strumenti di rete.

cAdvisor (Google) <https://github.com/google/cadvisor>

Monitoring risorse container in tempo reale.

Loki + Grafana <https://grafana.com/docs/loki/latest/>

Stack di log aggregation open source.

Linux nsenter <https://man7.org/linux/man-pages/man1/nsenter.1.html>

Man page di nsenter, strumento fondamentale per il debug.

Appendice A

Progetto Completo: Server Apache2 da Zero

Questa appendice costruisce un server HTTP Apache2 containerizzato partendo da zero, spiegando ogni scelta. Tutti i file sono stati verificati prima della pubblicazione.

A.1 Struttura del progetto

Listing A.1: Struttura delle directory del progetto

```
1  apache-docker/  
2  |-- Dockerfile  
3  |-- .dockerignore  
4  |-- compose.yaml  
5  |-- config/  
6  |   |-- apache2.conf          # configurazione principale  
7  |   '-- sites/  
8  |       '-- 000-default.conf # virtualhost di default  
9  |-- html/  
10 |   |-- index.html           # pagina principale  
11 |   |-- about.html  
12 |   '-- assets/  
13 |       |-- style.css  
14 |       '-- logo.png  
15 '-- logs/                   # cartella per i log (volume bind)  
16  
17 # Crea la struttura  
18 mkdir -p apache-docker/{config/sites,html/assets,logs}  
19 cd apache-docker
```

A.2 Passo 1 — La pagina HTML

Listing A.2: html/index.html

```
1  cat > html/index.html << 'HTML'  
2  <!DOCTYPE html>  
3  <html lang='it'>  
4  <head>  
5      <meta charset='UTF-8'>  
6      <meta name='viewport' content='width=device-width, initial-scale  
7          =1.0'>  
8      <title>Apache2 in Docker --- OTHack1A</title>  
9      <link rel='stylesheet' href='assets/style.css'>  
10 </head>  
11 <body>  
    <header>
```

```
12     <h1>Apache2 in Docker</h1>
13     <p>Containerizzato con Debian 12 Slim</p>
14 </header>
15 <main>
16     <section class='info'>
17         <h2>Informazioni Server</h2>
18         <ul>
19             <li>Web Server: Apache/2.4</li>
20             <li>Base Image: debian:12-slim</li>
21             <li>Autore: OTHack1A</li>
22         </ul>
23     </section>
24     <section class='links'>
25         <h2>Link utili</h2>
26         <ul>
27             <li><a href='/server-status'>Server Status</a></li>
28             <li><a href='/server-info'>Server Info</a></li>
29             <li><a href='about.html'>About</a></li>
30         </ul>
31     </section>
32 </main>
33 <footer>
34     <p>Documento personale --- OTHack1A 2026</p>
35 </footer>
36 </body>
37 </html>
38 HTML
```

Listing A.3: html/assets/style.css

```
1 cat > html/assets/style.css << 'CSS'
2 *, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }
3
4 body {
5     font-family: 'Segoe UI', system-ui, sans-serif;
6     background: #f5f7fa;
7     color: #222;
8     min-height: 100vh;
9     display: flex;
10    flex-direction: column;
11 }
12 header {
13     background: linear-gradient(135deg, #0a5c94, #0d7ac4);
14     color: white;
15     padding: 2rem;
16     text-align: center;
17 }
18 header h1 { font-size: 2rem; margin-bottom: 0.5rem; }
19 header p { opacity: 0.85; }
20 main {
21     flex: 1;
22     max-width: 800px;
23     margin: 2rem auto;
24     padding: 0 1rem;
25     display: grid;
26     grid-template-columns: 1fr 1fr;
27     gap: 1.5rem;
```

```
28 }
29 section {
30     background: white;
31     border-radius: 8px;
32     padding: 1.5rem;
33     box-shadow: 0 2px 8px rgba(0,0,0,0.08);
34 }
35 section h2 { color: #0a5c94; margin-bottom: 1rem; font-size: 1.2rem; }
36 ul { list-style: none; }
37 li { padding: 0.4rem 0; border-bottom: 1px solid #eee; }
38 a { color: #0a5c94; text-decoration: none; }
39 a:hover { text-decoration: underline; }
40 footer {
41     text-align: center;
42     padding: 1rem;
43     background: #2c3e50;
44     color: #aaa;
45     font-size: 0.85rem;
46 }
47 CSS
```

A.3 Passo 2 — Configurazione Apache2

Listing A.4: config/apache2.conf — configurazione principale

```
1 cat > config/apache2.conf << 'CONF'
2 # Configurazione minimale Apache2 per Docker
3 # Solo le direttive necessarie
4
5 ServerRoot "/etc/apache2"
6 PidFile /var/run/apache2/apache2.pid
7 Timeout 60
8 KeepAlive On
9 MaxKeepAliveRequests 100
10 KeepAliveTimeout 5
11
12 # Sicurezza: nascondi versione Apache
13 ServerTokens Prod
14 ServerSignature Off
15
16 # Carica solo i moduli necessari
17 LoadModule mpm_event_module /usr/lib/apache2/modules/mod_mpm_event.so
18 LoadModule authz_core_module /usr/lib/apache2/modules/mod_authz_core.so
19 LoadModule dir_module /usr/lib/apache2/modules/mod_dir.so
20 LoadModule mime_module /usr/lib/apache2/modules/mod_mime.so
21 LoadModule log_config_module /usr/lib/apache2/modules/mod_log_config.so
22 LoadModule unixd_module /usr/lib/apache2/modules/mod_unixd.so
23 LoadModule alias_module /usr/lib/apache2/modules/mod_alias.so
24 LoadModule status_module /usr/lib/apache2/modules/mod_status.so
25 LoadModule headers_module /usr/lib/apache2/modules/mod_headers.so
26
27 # Processo: gira come www-data (mai come root!)
28 User www-data
29 Group www-data
30
31 # Log su stdout/stderr per Docker (usa docker logs)
```

```
32 ErrorLog /proc/self/fd/2
33 CustomLog /proc/self/fd/1 combined
34
35 # Typo MIME
36 TypesConfig /etc/mime.types
37 DefaultType application/octet-stream
38
39 # Include VirtualHost
40 IncludeOptional /etc/apache2/sites-enabled/*.conf
41 CONF
```

Listing A.5: config/sites/000-default.conf — VirtualHost

```
1 cat > config/sites/000-default.conf << 'CONF'
2 <VirtualHost *:80>
3     ServerAdmin webmaster@localhost
4     DocumentRoot /var/www/html
5
6     # Header di sicurezza
7     Header always set X-Content-Type-Options 'nosniff'
8     Header always set X-Frame-Options 'DENY'
9     Header always set X-XSS-Protection '1; mode=block'
10    Header always set Referrer-Policy 'strict-origin-when-cross-origin'
11
12    <Directory /var/www/html>
13        Options -Indexes +FollowSymLinks
14        AllowOverride None
15        Require all granted
16        DirectoryIndex index.html index.htm
17    </Directory>
18
19    # Server Status (solo da localhost)
20    <Location /server-status>
21        SetHandler server-status
22        Require local
23    </Location>
24
25    # Log formato personalizzato
26    LogFormat '%h %l %u %t \"%r\" %s %b \"%{Referer}i\"' combined
27    CustomLog /proc/self/fd/1 combined
28    ErrorLog /proc/self/fd/2
29 </VirtualHost>
30 CONF
```

A.4 Passo 3 — Il Dockerfile

Listing A.6: Dockerfile — annotato linea per linea

```
1 # -- Scelta immagine base --
2 # debian:12-slim: senza componenti superflui (apt, dpkg, bash rimangono)
3 # NON usare :latest (non riproducibile)
4 FROM debian:12-slim
5
6 # -- Metadati OCI --
7 LABEL org.opencontainers.image.title='Apache2 HTTP Server' \
```

```
8      org.opencontainers.image.description='Apache2 su Debian 12 Slim' \
9      org.opencontainers.image.author='OTHack1A' \
10     org.opencontainers.image.version='1.0.0' \
11     org.opencontainers.image.url='https://github.com/OTHack1A'
12
13 # -- Installa Apache2 --
14 # Tutto in UN SOLO layer RUN per minimizzare i layer e la dimensione:
15 # - apt-get update + install nella stessa istruzione
16 # - --no-install-recommends: salta pacchetti opzionali
17 # - rm -rf /var/lib/apt/lists/*: rimuove cache apt (riduce ~30MB)
18 RUN apt-get update \
19     && apt-get install -y --no-install-recommends \
20         apache2=2.4.62-1~deb12u1 \
21         curl \
22     && apt-get clean \
23     && rm -rf /var/lib/apt/lists/* \
24     # Crea cartella PID (non esiste nella slim)
25     && mkdir -p /var/run/apache2 \
26     && chown www-data:www-data /var/run/apache2
27
28 # -- Copia configurazione --
29 # Usa COPY (non ADD): ADD e' solo per URL remoti e tar auto-extract
30 COPY config/apache2.conf /etc/apache2/apache2.conf
31 COPY config/sites/000-default.conf /etc/apache2/sites-enabled/000-default.conf
32
33 # -- Copia contenuto web --
34 COPY html/ /var/www/html/
35
36 # Imposta permessi corretti per www-data
37 RUN chown -R www-data:www-data /var/www/html \
38     && chmod -R 755 /var/www/html
39
40 # -- Porta --
41 # EXPOSE documenta la porta; la pubblicazione avviene con -p al run
42 EXPOSE 80
43
44 # -- Healthcheck --
45 # Il container viene marcato healthy/unhealthy in base al risultato
46 # --interval: ogni 30s controlla
47 # --timeout: 5s massimo per la risposta
48 # --start-period: aspetta 10s prima di iniziare i check (avvio lento)
49 # --retries: 3 fallimenti consecutivi -> unhealthy
50 HEALTHCHECK --interval=30s --timeout=5s \
51     --start-period=10s --retries=3 \
52     CMD curl -f http://localhost/ || exit 1
53
54 # -- Comando di avvio --
55 # ENTRYPOINT fisso + CMD come argomenti predefiniti (best practice)
56 # -D FOREGROUND: fondamentale per Docker (non andare in background!)
57 ENTRYPOINT ['apachectl']
58 CMD ['-D', 'FOREGROUND']
```

A.5 Passo 4 — .dockerignore

Listing A.7: `.dockerignore` — escludi file inutili dal contesto di build

```
1  # Controllo versione
2  .git/
3  .gitignore
4  .github/
5
6  # Documentazione
7  *.md
8  README*
9  CHANGELOG*
10
11 # Log e temporanei
12 logs/
13 *.log
14 *.tmp
15
16 # Ambienti di sviluppo
17 .vscode/
18 .idea/
19 *.swp
20 *~
21
22 # Segreti (MAI includere nel contesto di build)
23 .env
24 .env.*
25 *.pem
26 *.key
27 secrets/
28
29 # File OS
30 .DS_Store
31 Thumbs.db
```

A.6 Passo 5 — `compose.yaml`

Listing A.8: `compose.yaml` — orchestrazione completa

```
1  services:
2    apache:
3      build:
4        context: .
5        dockerfile: Dockerfile
6
7      # Nome del container (opzionale ma utile)
8      container_name: apache-demo
9
10     # Pubblica porta 80 del container sulla 8080 dell'host
11     # (porta < 1024 richiede root sull'host; usiamo 8080)
12     ports:
13       - '8080:80'
14
15     # Bind mount per i log (visibili sull'host in ./logs/)
16     volumes:
17       - ./logs:/var/log/apache2
18
```

```

19     # Riavvia automaticamente tranne se fermato manualmente
20     restart: unless-stopped
21
22     # Limiti risorse (buona pratica anche in sviluppo)
23     deploy:
24         resources:
25             limits:
26                 cpus: '0.25'
27                 memory: 128M
28             reservations:
29                 memory: 64M
30
31     # Healthcheck (rispecchia quello del Dockerfile)
32     healthcheck:
33         test: ['CMD', 'curl', '-f', 'http://localhost/']
34         interval: 30s
35         timeout: 5s
36         retries: 3
37         start_period: 10s
38
39     # Non espone variabili sensibili (Apache non ne ha bisogno)
40     # Se servissero:
41     # environment:
42     #     SERVER_ADMIN: webmaster@example.com
43     # oppure da file:
44     # env_file: .env

```

A.7 Passo 6 — Build, avvio e test

Listing A.9: Build, avvio e test completo

```

1  # -- BUILD --
2  # Costruisce l'immagine dal Dockerfile
3  docker compose build
4
5  # Con output verboso (utile per debug)
6  docker compose build --progress=plain
7
8  # -- AVVIO --
9  docker compose up -d
10
11 # Verifica stato
12 docker compose ps
13 # NAME                IMAGE                COMMAND                STATUS
14 # apache-demo         apache-docker-a     'apachectl -D FOREG\ldots{}' Up (healthy)
15 # 0.0.0.0:8080->80/tcp
16
17 # -- TEST --
18 # Test HTTP base
19 curl -i http://localhost:8080/
20 # HTTP/1.1 200 OK
21 # Server: Apache
22 # Content-Type: text/html
23 # ...

```



```

24 # Test healthcheck manuale
25 curl -f http://localhost:8080/ && echo 'OK' || echo 'FAIL'
26
27 # Headers di sicurezza
28 curl -I http://localhost:8080/ | grep -E 'X-Content|X-Frame|X-XSS'
29 # X-Content-Type-Options: nosniff
30 # X-Frame-Options: DENY
31 # X-XSS-Protection: 1; mode=block
32
33 # -- LOG --
34 # Log in tempo reale (Docker streams stdout/stderr)
35 docker compose logs -f apache
36
37 # Log su file (volume bind in ./logs/)
38 ls -la logs/
39 cat logs/access.log # se Apache e' configurato su file
40
41 # -- ISPEZIONE --
42 # Entra nel container (bash disponibile in debian:12-slim)
43 docker compose exec apache bash
44 # root@abc123:/# apache2 -v
45 # Server version: Apache/2.4.62 (Debian)
46
47 # Processi in esecuzione nel container
48 docker compose exec apache ps aux
49 # USER      PID  COMMAND
50 # root       1   apachectl -D FOREGROUND <- PID 1: il processo padre
51 # www-data   8   /usr/sbin/apache2 -D FOREGROUND <- worker processes
52 # www-data   9   /usr/sbin/apache2 -D FOREGROUND
53 # www-data  10   /usr/sbin/apache2 -D FOREGROUND
54
55 # Dimensione finale dell'immagine
56 docker image ls apache-docker-apache
57 # REPOSITORY          TAG      SIZE
58 # apache-docker-apache latest  87.3MB  <- molto piu' piccola di httpd
59 # ufficiale!
60
61 # Analisi layer (opzionale, richiede dive)
62 # dive apache-docker-apache:latest
63
64 # -- AGGIORNAMENTO CONTENUTO (senza rebuild) --
65 # Il bind mount in compose.yaml aggiorna i log automaticamente
66 # Per aggiornare l'HTML basta copiare e ricaricare:
67 cp nuova_pagina.html html/
68 docker compose restart apache
69 # oppure ricostruisci l'immagine:
70 docker compose up -d --build
71
72 # -- PULIZIA --
73 docker compose down
74 docker compose down --rmi local # rimuove anche l'immagine buildata

```

A.8 Passo 7 — Variante con HTTPS (TLS)

Listing A.10: Aggiunta di HTTPS con certificato self-signed

```

1 # Genera certificato self-signed (valido 365 giorni)
2 mkdir -p certs
3 openssl req -x509 -newkey rsa:4096 -sha256 \
4     -keyout certs/server.key \
5     -out     certs/server.crt \
6     -days   365 -nodes \
7     -subj    '/C=IT/ST=Emilia-Romagna/L=Cesena/CN=localhost'

```

Listing A.11: Dockerfile con SSL abilitato

```

1 FROM debian:12-slim
2
3 LABEL org.opencontainers.image.title='Apache2 HTTPS'
4
5 RUN apt-get update \
6     && apt-get install -y --no-install-recommends \
7         apache2 curl openssl \
8     && apt-get clean \
9     && rm -rf /var/lib/apt/lists/* \
10    # Abilita moduli SSL e rewrite
11    && a2enmod ssl rewrite headers \
12    && mkdir -p /var/run/apache2 \
13    && chown www-data:www-data /var/run/apache2
14
15 COPY config/apache2.conf /etc/apache2/apache2.conf
16 COPY config/sites/000-default-ssl.conf /etc/apache2/sites-enabled/
17 COPY html/ /var/www/html/
18 COPY certs/server.crt /etc/apache2/ssl/server.crt
19 COPY certs/server.key /etc/apache2/ssl/server.key
20
21 RUN chown -R www-data:www-data /var/www/html /etc/apache2/ssl
22
23 EXPOSE 80 443
24
25 HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
26     CMD curl -fk https://localhost/ || exit 1
27
28 ENTRYPOINT ['apachectl']
29 CMD ['-D', 'FOREGROUND']

```

Listing A.12: config/sites/000-default-ssl.conf

```

1 cat > config/sites/000-default-ssl.conf << 'CONF'
2 # Redirect HTTP -> HTTPS
3 <VirtualHost *:80>
4     RewriteEngine On
5     RewriteRule ^(.*)$ https://%{HTTP_HOST}$1 [R=301,L]
6 </VirtualHost>
7
8 # VirtualHost HTTPS
9 <VirtualHost *:443>
10     ServerAdmin webmaster@localhost
11     DocumentRoot /var/www/html
12
13     SSLEngine on
14     SSLCertificateFile /etc/apache2/ssl/server.crt

```

```

15     SSLCertificateKeyFile /etc/apache2/ssl/server.key
16
17     # Configura TLS moderno
18     SSLProtocol -all +TLSv1.2 +TLSv1.3
19     SSLCipherSuite ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-
        SHA256
20
21     Header always set Strict-Transport-Security \
22         'max-age=31536000; includeSubDomains'
23     Header always set X-Content-Type-Options 'nosniff'
24     Header always set X-Frame-Options 'DENY'
25
26     <Directory /var/www/html>
27         Options -Indexes +FollowSymLinks
28         AllowOverride None
29         Require all granted
30     </Directory>
31
32     CustomLog /proc/self/fd/1 combined
33     ErrorLog /proc/self/fd/2
34 </VirtualHost>
35 CONF

```

Listing A.13: compose.yaml aggiornato per HTTPS

```

1 services:
2   apache:
3     build: .
4     container_name: apache-https
5     ports:
6       - '8080:80'
7       - '8443:443'
8     volumes:
9       - ./logs:/var/log/apache2
10    restart: unless-stopped
11    healthcheck:
12      test: ['CMD', 'curl', '-fk', 'https://localhost/']
13      interval: 30s
14      timeout: 5s
15      retries: 3
16      start_period: 15s

```

Listing A.14: Test HTTPS

```

1 # Rebuilda e avvia
2 docker compose up -d --build
3
4 # Test HTTPS (-k = ignora self-signed)
5 curl -kI https://localhost:8443/
6 # HTTP/2 200
7 # strict-transport-security: max-age=31536000; includeSubDomains
8 # x-content-type-options: nosniff
9
10 # Test redirect HTTP -> HTTPS
11 curl -I http://localhost:8080/
12 # HTTP/1.1 301 Moved Permanently
13 # Location: https://localhost/

```

A.9 Riepilogo: cosa abbiamo costruito

Tabella A.1: Riepilogo del progetto Apache2 Docker

Elemento	Scelta e motivazione
Base image	<code>debian:12-slim</code> : piccola, stabile, con apt disponibile
Installazione	RUN unico con <code>-no-install-recommends</code> + pulizia cache
Utente	<code>www-data</code> (non root) per il processo Apache
Log	Redirect su <code>/proc/self/fd/1,2</code> per <code>docker logs</code>
Healthcheck	<code>curl -f localhost/</code> con parametri realistici
Avvio	ENTRYPOINT + CMD con <code>-D FOREGROUND</code>
Sicurezza	<code>ServerTokens Prod</code> , header <code>X-*</code> , nessun Directory listing
Dimensione	~87 MB vs ~200 MB di <code>httpd:latest</code>

Appendice B

Cheatsheet Comandi Docker

B.1 Immagini

Tabella B.1: Comandi per la gestione delle immagini

Comando	Funzione
<code>docker images</code>	Lista immagini locali
<code>docker pull nginx:1.25-alpine</code>	Scarica immagine dal registry
<code>docker build -t app:1.0 .</code>	Costruisce immagine dal Dockerfile
<code>docker build -no-cache -t app .</code>	Build senza cache
<code>docker tag app:1.0 registry/app:1.0</code>	Rinomina/tagga immagine
<code>docker push registry/app:1.0</code>	Pubblica immagine sul registry
<code>docker image history app:1.0</code>	Mostra i layer dell'immagine
<code>docker image inspect app:1.0</code>	Metadati completi in JSON
<code>docker rmi app:1.0</code>	Rimuovi immagine
<code>docker image prune -a</code>	Rimuovi tutte le immagini non usate

B.2 Container

Tabella B.2: Comandi per la gestione dei container

Comando	Funzione
<code>docker run -d -p 8080:80 nginx</code>	Avvia in background con port mapping
<code>docker run -it ubuntu bash</code>	Avvia in modalità interattiva
<code>docker run -rm alpine echo hi</code>	Avvia ed elimina al termine
<code>docker run -name myapp -d app</code>	Avvia con nome personalizzato
<code>docker ps</code>	Lista container in esecuzione
<code>docker ps -a</code>	Lista tutti i container (anche fermi)
<code>docker stop myapp</code>	Ferma il container (SIGTERM)
<code>docker kill myapp</code>	Forza stop (SIGKILL)
<code>docker rm myapp</code>	Rimuovi container fermo
<code>docker rm -f myapp</code>	Forza rimozione anche se in esecuzione
<code>docker restart myapp</code>	Riavvia
<code>docker exec -it myapp bash</code>	Shell interattiva nel container
<code>docker exec myapp ls /app</code>	Esegui comando senza shell interattiva
<code>docker logs myapp</code>	Mostra log
<code>docker logs -f myapp</code>	Segui log in tempo reale
<code>docker logs -tail=50 myapp</code>	Ultime 50 righe
<code>docker inspect myapp</code>	Metadati completi JSON
<code>docker stats</code>	Monitor risorse in tempo reale
<code>docker top myapp</code>	Processi nel container
<code>docker cp myapp:/file.txt .</code>	Copia file dal container all'host
<code>docker cp ./file.txt myapp:/</code>	Copia file dall'host al container
<code>docker container prune</code>	Rimuovi tutti i container fermi

B.3 Volumi e Reti

Tabella B.3: Comandi per volumi e reti

Comando	Funzione
<code>docker volume create mydata</code>	Crea volume named
<code>docker volume ls</code>	Lista volumi
<code>docker volume inspect mydata</code>	Dettagli volume
<code>docker volume rm mydata</code>	Rimuovi volume
<code>docker volume prune</code>	Rimuovi volumi non usati
<code>docker network create mynet</code>	Crea rete bridge
<code>docker network ls</code>	Lista reti
<code>docker network inspect mynet</code>	Dettagli rete
<code>docker network connect mynet c1</code>	Connetti container alla rete
<code>docker network disconnect mynet c1</code>	Disconnetti
<code>docker network rm mynet</code>	Rimuovi rete
<code>docker network prune</code>	Rimuovi reti non usate

B.4 Docker Compose

Tabella B.4: Comandi Docker Compose v2/v5 essenziali

Comando	Funzione
<code>docker compose up -d</code>	Avvia tutti i servizi in background
<code>docker compose up -d --build</code>	Forza rebuild prima dell'avvio
<code>docker compose down</code>	Ferma e rimuove container e reti
<code>docker compose down -v</code>	Rimuovi anche i volumi
<code>docker compose ps</code>	Stato dei servizi
<code>docker compose logs -f</code>	Log di tutti i servizi
<code>docker compose logs -f web</code>	Log di un servizio specifico
<code>docker compose exec web bash</code>	Shell in container attivo
<code>docker compose run --rm web sh</code>	Nuovo container temporaneo
<code>docker compose restart web</code>	Riavvia un servizio
<code>docker compose pull</code>	Aggiorna le immagini
<code>docker compose build</code>	Rebuilda immagini locali
<code>docker compose stop</code>	Ferma senza rimuovere
<code>docker compose start</code>	Avvia container esistenti
<code>docker compose scale web=3</code>	Scala a 3 istanze
<code>docker compose watch</code>	Hot-reload sviluppo

B.5 Pulizia sistema

Listing B.1: Comandi di pulizia (da usare con attenzione)

```
1 # Mostra spazio usato da Docker
2 docker system df
3
4 # Output:
5 # TYPE                TOTAL    ACTIVE    SIZE      RECLAIMABLE
6 # Images               23       5         4.123GB   3.456GB (83%)
7 # Containers           3        3         12.5kB    0B
8 # Local Volumes        8        4         2.345GB   1.234GB (52%)
9 # Build Cache          145      0         3.456GB   3.456GB
10
11 # Rimuovi risorse non usate (container fermi, reti orfane,
12 # immagini dangling, cache build)
13 docker system prune
14
15 # Rimuovi TUTTO (incluse immagini non usate e volumi)
16 docker system prune -a --volumes # ATTENZIONE: irreversibile!
17
18 # Pulizia selettiva
19 docker container prune # container fermi
20 docker image prune     # immagini dangling (<none>)
21 docker image prune -a  # tutte le immagini non usate
22 docker volume prune    # volumi non montati
23 docker network prune   # reti non usate
24 docker builder prune   # cache build
25
26 # Pulisci build cache piu' vecchia di 24h
27 docker builder prune --filter until=24h
```



Appendice C

Fonti e Riferimenti

C.1 Documentazione ufficiale

Docker Docs <https://docs.docker.com>

Documentazione ufficiale: Engine, Compose, BuildKit, Desktop.

Docker Blog <https://www.docker.com/blog>

Annunci, novità e approfondimenti tecnici dal team Docker.

Compose Specification <https://compose-spec.io>

Specifiche formali del formato `compose.yaml`.

OCI (Open Container Initiative) <https://opencontainers.org>

Standard OCI per runtime (runc), immagini e distribuzione.

containerd documentation <https://containerd.io/docs>

Documentazione del runtime containerd 2.0.

NVIDIA Container Toolkit <https://docs.nvidia.com/datacenter/cloud-native/container-toolkit>

Guida ufficiale al toolkit GPU per container.

C.2 Articoli e risorse online

Compose evolution (Docker Docs) <https://docs.docker.com/compose/intro/history/>

Storia completa di Docker Compose v1, v2, v5. Aggiornato a marzo 2026.

Podman e containerd 2.0 nel 2026 (DEV Community) <https://dev.to/dataformathub/deep-dive-why-podman-and-containerd-20-are-replacing-docker-in-2026-32ak>

Containerizzazione 2025 (DEV Community) <https://dev.to/dataformathub/containerization-2025>

Docker AI Catalog 2026 <https://dasroot.net/posts/2025/12/docker-ai-catalog-genera-images-a>

Docker GPU e LLM (2026) <https://dasroot.net/posts/2026/01/docker-gpu-support-llms-contai>

Docker e NVIDIA GenAI Stack (Docker Blog) <https://www.docker.com/blog/docker-nvidia-sup>

Top 5 Docker Container per LLM (USDSI) <https://www.usdsi.org/data-science-insights/top-5-docker-containers-transforming-llm-development-in-2026>

C.3 Libri consigliati

Docker Deep Dive (2024 Edition) Nigel Poulton. Poulton Publishing, 2024. Aggiornato a Docker Engine 25.x e Compose v2.

Docker in Practice (3rd Edition) Ian Miell, Aidan Hobson Sayers. Manning, 2024.

Container Security Liz Rice. O'Reilly, 2020. Fondamentale per capire cgroups, namespace e sicurezza container.

Kubernetes in Action (2nd Edition) Marko Luksa. Manning, 2024. Per chi vuole passare da Compose a Kubernetes.

C.4 Fonti per sicurezza e diagnostica

Trivy (vulnerability scanner) <https://trivy.dev>

Scanner di vulnerabilit  per immagini Docker, filesystem e git repo.

Hadolint (Dockerfile linter) <https://github.com/hadolint/hadolint>

Linter per Dockerfile che controlla best practice e sicurezza.

Cosign (image signing) <https://github.com/sigstore/cosign>

Firma e verifica immagini OCI con Sigstore.

Syft (SBOM generation) <https://github.com/anchore/syft>

Genera Software Bill of Materials per immagini e filesystem.

Grype (vulnerability scanner da SBOM) <https://github.com/anchore/grype>

Scansiona SBOM per vulnerabilit  note.

Docker Security Benchmarks (CIS) <https://www.cisecurity.org/benchmark/docker>

Linee guida CIS per hardening Docker.

OWASP Docker Security https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html

Cheatsheet OWASP per la sicurezza Docker.

Traefik Proxy <https://traefik.io/traefik/>

Reverse proxy cloud-native con integrazione Docker nativa.

Dive (layer analyzer) <https://github.com/wagoodman/dive>

Tool per analizzare i layer di immagini Docker e ottimizzarne la dimensione.

Glossario

Termini tecnici usati nel documento. Per ciascuno: **significato**, **dove viene usato** e **funzioni principali**.

ARG *Significato:* Istruzione Dockerfile che definisce una variabile disponibile solo durante la fase di build, non nel container a runtime. *Dove:* Nel Dockerfile; passata con `docker build --build-arg`. *Funzioni:* Parametrizzare la build (versioni, ambienti) senza esporre valori sensibili nel container. Attenzione: visibile in `docker history`; non usare per segreti.

Bind mount *Significato:* Montaggio di una directory dell'host all'interno del container, creando una corrispondenza diretta tra i due percorsi. *Dove:* Con `-v /host/path:/container/path` o nel campo `volumes:` di `compose.yaml`. *Funzioni:* Sviluppo con hot-reload (i file modificati sull'host sono immediatamente visibili nel container), condivisione di configurazioni, lettura di socket Unix (`/var/run/docker.sock`).

BuildKit *Significato:* Motore di build avanzato introdotto da Docker Inc. e diventato default con Docker 23.0 (2023). *Dove:* Usato da `docker build` e `docker buildx`; il daemon è `buildkitd`. *Funzioni:* Build parallele, cache avanzata (locale e da registry remoto), multi-platform (`linux/amd64`, `linux/arm64`), secrets sicuri che non entrano nei layer, cache-mount per gestori pacchetti.

cgroups (Control Groups) *Significato:* Funzionalità del kernel Linux che permette di raggruppare processi e applicare limiti di utilizzo delle risorse. *Dove:* Usati da Docker per ogni container; visibili in `/sys/fs/cgroup/`; cgroups v2 è il default su Debian 12 e Arch Linux. *Funzioni:* Limitano CPU (`-cpus`), memoria (`-memory`), I/O disco e numero di processi (`pids.max`). Senza cgroups i container potrebbero consumare risorse illimitate.

CMD *Significato:* Istruzione Dockerfile che specifica il comando predefinito eseguito all'avvio del container. *Dove:* Come ultima istruzione del Dockerfile; sovrascrivibile passando un comando a `docker run`. *Funzioni:* Quando usato insieme a `ENTRYPOINT`, `CMD` fornisce gli argomenti predefiniti. Esistono tre forme: exec form (`[“cmd“,“arg“]`), shell form (`cmd arg`) e come argomenti di `ENTRYPOINT`.

containerd *Significato:* Runtime di container di alto livello che gestisce il ciclo di vita completo: pull immagini, creazione, esecuzione, networking e storage. *Dove:* Componente centrale di Docker Engine; usato anche direttamente da Kubernetes come CRI (Container Runtime Interface). *Funzioni:* Gestisce gli OCI bundle, chiama `runc` per creare i container, gestisce snapshotter e content store. La versione 2.0 (2024) introduce hardening sicurezza e performance migliorate.

COPY vs ADD *Significato:* Entrambe le istruzioni Dockerfile copiano file nel filesystem

dell'immagine, ma con comportamenti diversi. *Dove:* Nel Dockerfile per includere file del contesto di build. *Funzioni:* COPY è semplice e preferita: copia file/directory. ADD fa lo stesso più: scarica URL remoti e decompone automaticamente archivi tar. Raccomandazione ufficiale: usare sempre COPY, ADD solo quando serve la decompressione automatica.

Docker Compose *Significato:* Tool per definire e gestire applicazioni multi-container tramite un file YAML dichiarativo. *Dove:* File `compose.yaml` (o `docker-compose.yml`); invocato con `docker compose` (senza trattino, versione Go v2/v5). *Funzioni:* Definisce servizi, reti e volumi; gestisce dipendenze tra servizi (`depends_on`), variabili d'ambiente, healthcheck e limiti di risorse. Compose v1 (Python, con trattino) è stato rimosso ad aprile 2025.

Docker Hub *Significato:* Registry pubblico di immagini Docker, gestito da Docker Inc., con oltre 8 milioni di immagini disponibili. *Dove:* <https://hub.docker.com>; è il registry di default per `docker pull` e `docker push`. *Funzioni:* Ospita immagini ufficiali (nginx, postgres, python...), immagini verificate e immagini della community. Offre build automatizzate da GitHub/GitLab, webhooks e vulnerability scanning con Docker Scout.

Dockerfile *Significato:* File di testo con istruzioni sequenziali per costruire un'immagine Docker in modo riproducibile. *Dove:* Nella directory del progetto; passato a `docker build`. *Funzioni:* Definisce l'immagine base (FROM), installa dipendenze (RUN), copia file (COPY), espone porte (EXPOSE), configura l'ambiente (ENV) e specifica il comando di avvio (CMD/ENTRYPOINT). Ogni istruzione crea un layer dell'immagine.

ENTRYPOINT *Significato:* Istruzione Dockerfile che definisce il comando principale fisso del container, non sovrascrivibile senza `-entrypoint`. *Dove:* Nel Dockerfile; combinato con CMD per i parametri. *Funzioni:* Pattern comune: `ENTRYPOINT ["nginx"] + CMD ["-g", "daemon off;"]`; l'utente può sovrascrivere CMD (`docker run img -c custom.conf`) ma non ENTRYPOINT. Ideale per container con un singolo scopo eseguibile.

ENV *Significato:* Istruzione Dockerfile che imposta variabili d'ambiente disponibili sia durante la build che a runtime nel container. *Dove:* Nel Dockerfile; visibile con `docker inspect`; sovrascrivibile con `docker run -e VAR=valore`. *Funzioni:* Configurazione portabile dell'applicazione. Attenzione: i valori sono visibili con `docker inspect`; non usare per segreti. Usare invece secrets BuildKit o Docker Swarm secrets.

EXPOSE *Significato:* Istruzione Dockerfile che documenta le porte su cui il container ascolterà, senza pubblicarle effettivamente. *Dove:* Nel Dockerfile; la pubblicazione reale avviene con `-p` in `docker run` o `ports` in Compose. *Funzioni:* Documentazione per gli sviluppatori e per tool di orchestrazione. Con `docker run -P` pubblica automaticamente tutte le porte EXPOSE su porte casuali dell'host.

HEALTHCHECK *Significato:* Istruzione Dockerfile (o opzione Compose) che configura un comando periodico per verificare la salute del container. *Dove:* Nel Dockerfile o in `compose.yaml`; monitorabile con `docker ps` (colonna STATUS: healthy/unhealthy). *Funzioni:* Permette a Docker e agli orchestratori (Compose, Swarm, Kubernetes) di sapere se il container sta funzionando correttamente, non solo se il processo è vivo. Con `depends_on: condition:`

`service_healthy` in Compose, garantisce l'ordine di avvio.

Immagine Docker *Significato:* Template immutabile e portabile per creare container. Composta da layer sovrapposti in sola lettura. *Dove:* Memorizzata localmente in `/var/lib/docker/`; distribuita tramite registry OCI (Docker Hub, ghcr.io, registry privati). *Funzioni:* Contiene sistema operativo base, dipendenze, codice applicativo e configurazione. Identificata da `nome:tag` (es: `nginx:1.25-alpine`) o digest SHA256. Ogni `docker run` crea un container scrivibile sopra l'immagine read-only.

Layer *Significato:* Unità immutabile che compone un'immagine Docker; ogni istruzione `RUN/COPY/ADD` nel Dockerfile crea un nuovo layer. *Dove:* Memorizzati in `/var/lib/docker/overlay2/`; visibili con `docker image history`. *Funzioni:* I layer sono condivisi tra immagini diverse che usano la stessa base: risparmio di spazio e banda. Il layer superiore del container è l'unico scrivibile (copy-on-write). Ordine dei layer è critico per la cache: mettere i file che cambiano raramente (dipendenze) prima di quelli che cambiano spesso (codice sorgente).

Multi-stage build *Significato:* Tecnica Dockerfile che usa più istruzioni `FROM` per separare l'ambiente di build dall'immagine di runtime finale. *Dove:* Nel Dockerfile con più blocchi `FROM ... AS stage_name`. *Funzioni:* Riduce drasticamente la dimensione dell'immagine finale: il builder può pesare GB (con compilatori, SDK, test), mentre il runtime pesa pochi MB (solo i binari compilati). Esempio: Go app da 300 MB a 8 MB con `FROM scratch`.

Namespace *Significato:* Funzionalità del kernel Linux che isola le risorse di sistema tra gruppi di processi. *Dove:* Usati da Docker per ogni container; visibili in `/proc/PID/ns/`. *Funzioni:* I container hanno namespace separati per PID (albero dei processi), net (interfacce di rete), mnt (filesystem), uts (hostname), ipc (code messaggi) e user (UID/GID mapping). Questo fa sì che un processo nel container veda solo le risorse del proprio namespace.

NVIDIA Container Toolkit *Significato:* Set di strumenti che permette ai container Docker di accedere alle GPU NVIDIA senza installare driver CUDA dentro il container. *Dove:* Installato sull'host; configura Docker con il runtime `nvidia`; usato con `docker run --gpus all`. *Funzioni:* Fa da bridge tra il container e i driver NVIDIA dell'host. Il container vede la GPU come dispositivo nativo. Fondamentale per workload AI/ML: training e inference di LLM, reti neurali, computer vision.

OCI (Open Container Initiative) *Significato:* Organizzazione che definisce standard aperti per container: Runtime Specification, Image Specification e Distribution Specification. *Dove:* Fondato nel 2015 da Docker, CoreOS e altri. Gli standard sono implementati da runc, containerd, Docker, Podman. *Funzioni:* Garantisce che immagini e container siano interoperabili tra tool diversi (Docker, Podman, Kubernetes CRI). Un'immagine OCI costruita con Docker può girare con Podman o Kubernetes senza modifiche.

overlay2 *Significato:* Storage driver predefinito di Docker su Linux, basato sul filesystem overlay del kernel. *Dove:* In `/var/lib/docker/overlay2/`; configurabile in `/etc/docker/daemon.json`. *Funzioni:* Implementa il meccanismo copy-on-write dei layer: i layer read-only dell'immagine sono montati come `lowerdir`, lo strato scrivibile del container come `upperdir`; il container

vede la vista unificata in `merged/`.

Registry *Significato:* Servizio per archiviare e distribuire immagini Docker/OCI, accessibile tramite API HTTP standardizzata (OCI Distribution Spec). *Dove:* Docker Hub (default), GitHub Container Registry (ghcr.io), Google Container Registry (gcr.io), registry privato con `registry:2`. *Funzioni:* Ospita immagini taggate; permette push/pull autenticato; supporta webhook per CI/CD. Il registry privato locale si avvia in pochi secondi: `docker run -p 5000:5000 registry:2`.

runc *Significato:* Runtime OCI di basso livello che esegue effettivamente il container: crea namespace, imposta cgroups e avvia il processo PID 1. *Dove:* Chiamato da containerd-shim; scritto in Go/C; sviluppato da Docker Inc. e donato all'OCI. *Funzioni:* Implementa la OCI Runtime Specification. **runc** non sa nulla di immagini o registry: riceve un "OCI bundle" (filesystem + config.json) e lo esegue. Alternative: crun (più veloce, in C), gVisor runsc (sandbox sicura).

UnionFS *Significato:* Tipo di filesystem che sovrappone più directory (layer) presentandole come un'unica vista unificata. *Dove:* Implementato in Docker tramite overlay2 (kernel OverlayFS). *Funzioni:* Permette la condivisione di layer read-only tra immagini diverse (risparmio spazio) e la scrittura copy-on-write nel layer superiore del container. Quando un container modifica un file del layer read-only, viene prima copiato nel layer scrivibile (*copy-up*), poi modificato.

VOLUME *Significato:* Istruzione Dockerfile che dichiara un punto di mount per dati persistenti fuori dall'immagine. *Dove:* Nel Dockerfile; alla creazione del container Docker crea automaticamente un volume anonimo se non specificato. *Funzioni:* I dati nel volume sopravvivono all'eliminazione del container. In produzione, preferire volumi named (creati esplicitamente con `docker volume create`) ai volumi anonimi.

WORKDIR *Significato:* Istruzione Dockerfile che imposta la directory di lavoro per le istruzioni successive (RUN, CMD, COPY, ADD, ENTRYPOINT). *Dove:* Nel Dockerfile; la directory viene creata se non esiste. *Funzioni:* Evita di dover specificare percorsi assoluti in ogni istruzione. Best practice: usare sempre WORKDIR invece di `RUN cd /app` (che non persiste tra layer RUN).